

CSS Flexible Box Module: Guia de Arquitetura de Layout

O CSS Flexbox (**Flexible Box Layout**) é um modelo de layout **unidimensional**, projetado para organizar elementos **em linha ou em coluna**, de forma flexível, responsiva e eficiente, mesmo quando os tamanhos dos itens ou do contêiner são desconhecidos ou variáveis.

O conceito fundamental: a analogia da água



Qual é o formato da água? Vai depender do recipiente.

Esse é exatamente o princípio do Flexbox.

Assim como a água **não possui forma própria** e se adapta ao **contêiner** — seja um jarro, uma garrafa ou um copo — os **flex items** se ajustam dinamicamente ao **flex container**.

- Se houver **espaço disponível**, eles podem **crescer**
- Se o espaço for limitado, eles podem **encolher**
- Se a direção mudar, eles **reorganizam-se automaticamente**

No Flexbox, **o contêiner define o comportamento**, e os itens se moldam a ele.

Estrutura do Flexbox

Um layout Flexbox sempre possui:

Flex Container (o recipiente)

O elemento pai, que define:

```
display: flex;
```

Ele é o “jarro” que controla:

- direção (row ou column)
- alinhamento
- distribuição do espaço

☐ Flex Items (a água)

Os elementos filhos diretos:

- adaptam-se ao espaço disponível
- respeitam regras como `flex-grow`, `flex-shrink` e `flex-basis`
- fluem conforme o contêiner muda de tamanho

Flexbox vs Grid (na lógica da analogia)

- **Flexbox** → como água fluindo **em uma direção por vez** (linha **ou** coluna)
- **Grid** → como uma forma rígida de gelo (linhas **e** colunas ao mesmo tempo)

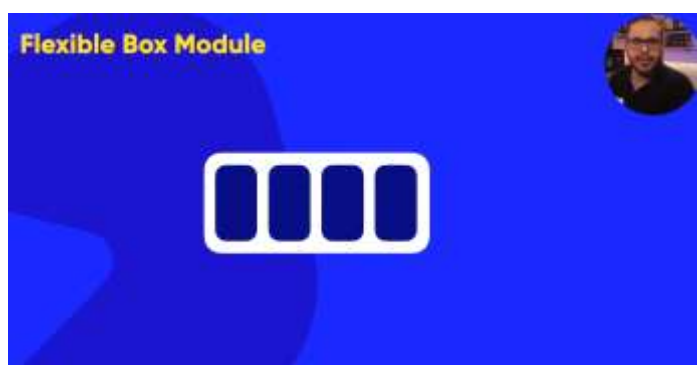
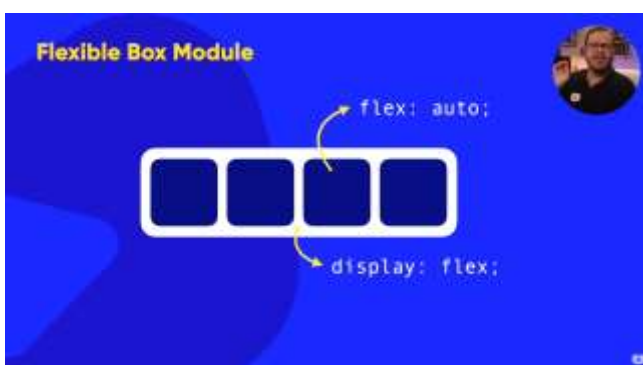
Por que usar Flexbox?

Porque ele permite criar layouts:

- flexíveis
- responsivos
- previsíveis
- sem uso de `float` ou posicionamento complexo

Assim como a água encontra equilíbrio dentro de qualquer recipiente, o Flexbox encontra o melhor arranjo possível dentro do espaço disponível.

2. Inicialização do Contexto Flex



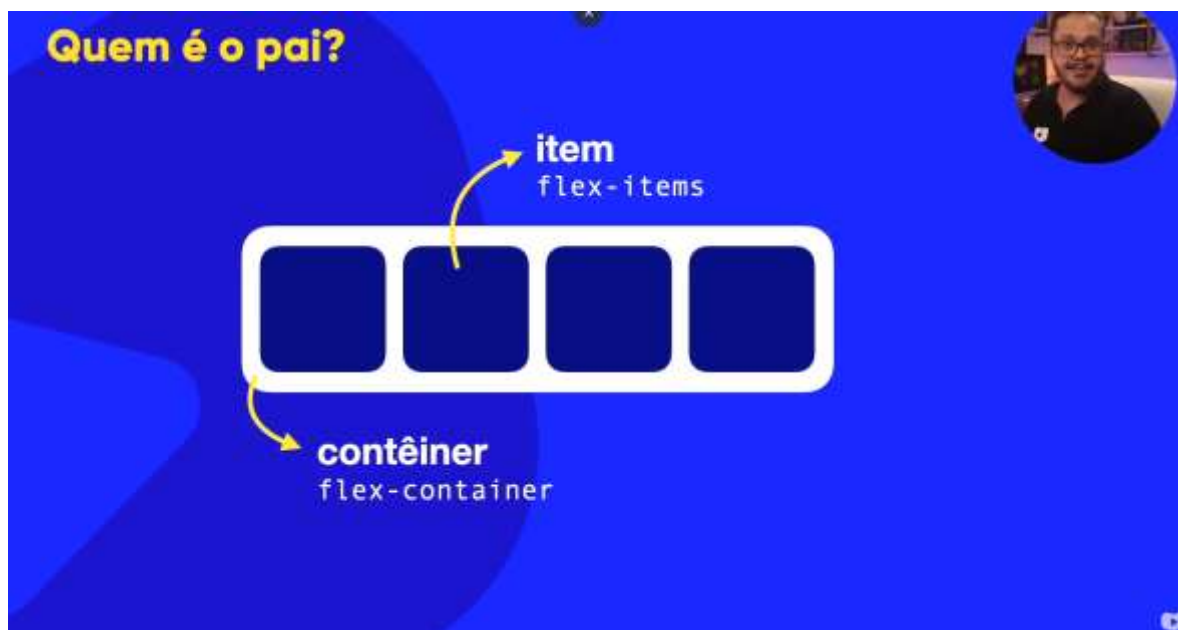
Para ativar este modelo de layout, definimos a propriedade `display` no elemento pai (container)

- `display: flex;`

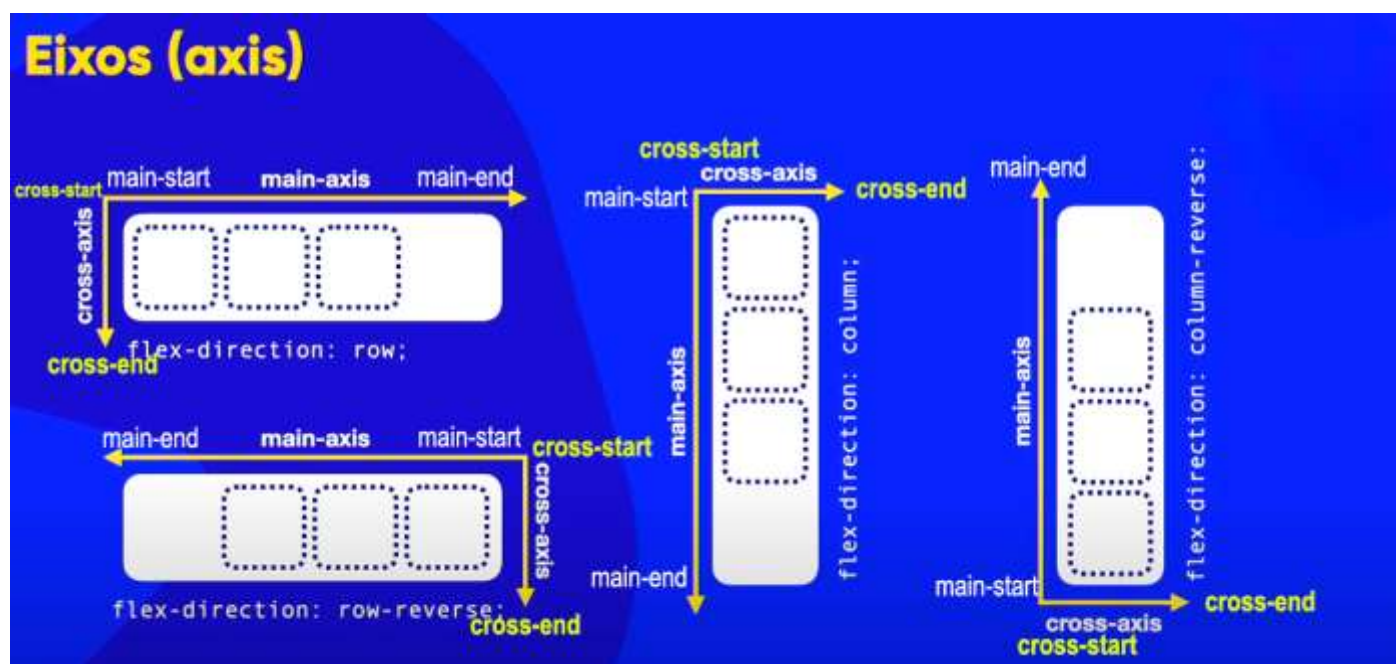
Transforma o elemento em um contêiner flexível de bloco. Seus filhos diretos tornam-se flex items.

- `display: inline-flex;`

O contêiner comporta-se como um elemento inline, mas mantém as propriedades flexíveis internamente.



3. Gerenciamento do Eixo Principal (Main Axis)



O controle de fluxo é determinado pela direção em que os itens são renderizados.

Define o vetor principal de distribuição.

- **row (Padrão):** Esquerda para a direita (em idiomas LTR).



- **row-reverse:** Direita para a esquerda.



- **column:** Topo para baixo.

flex-direction

flex-direction: column
é de cima para baixo

`flex-direction:
column;`



`flex-direction:
column;`



- **column-reverse:** Baixo para o topo.

flex-direction

flex-direction: column-reverse
inverte (de baixo para cima)

`flex-direction:
column-reverse`



`flex-direction:
column-reverse;`



Propriedade: *justify-content*

Gerencia o alinhamento e a distribuição de espaço ao longo do eixo principal (definido pelo `flex-direction`).

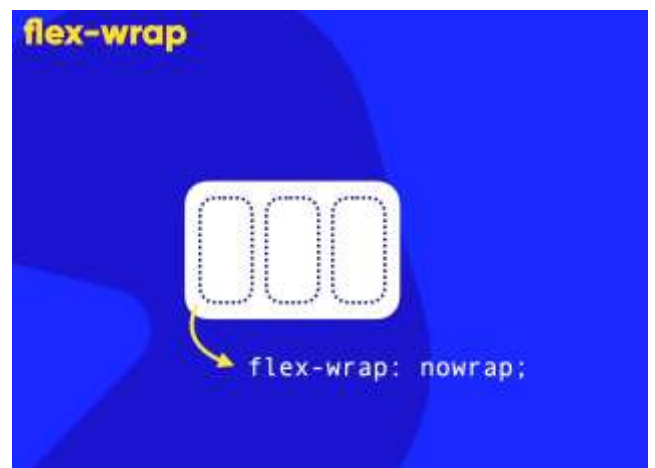
Valor	Comportamento
flex-start	Agrupar itens no início do contêiner (Padrão).
flex-end	Agrupar itens no final do contêiner.
center	Centraliza os itens no eixo principal.
space-between	Distribui itens uniformemente; o primeiro fica no início, o último no final.
space-around	Distribuição uniforme com espaçamento igual ao redor de cada item.
space-evenly	Espaçamento visualmente idêntico entre todos os itens e as bordas.

4. Controle de Quebra de Linha (Wrapping) - Empacotamento

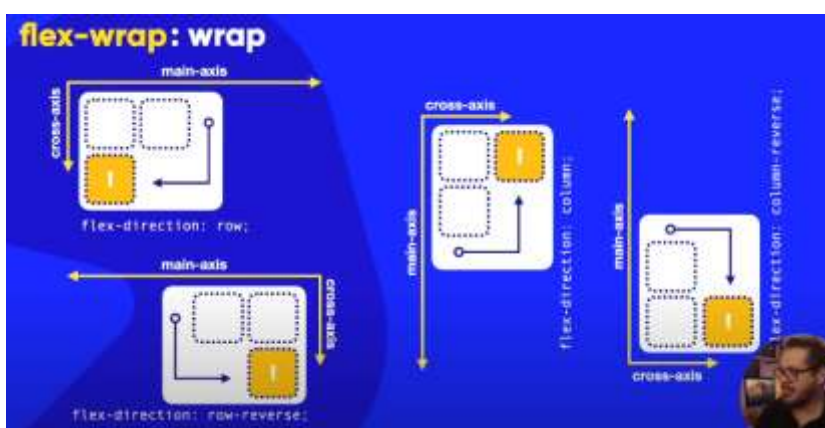
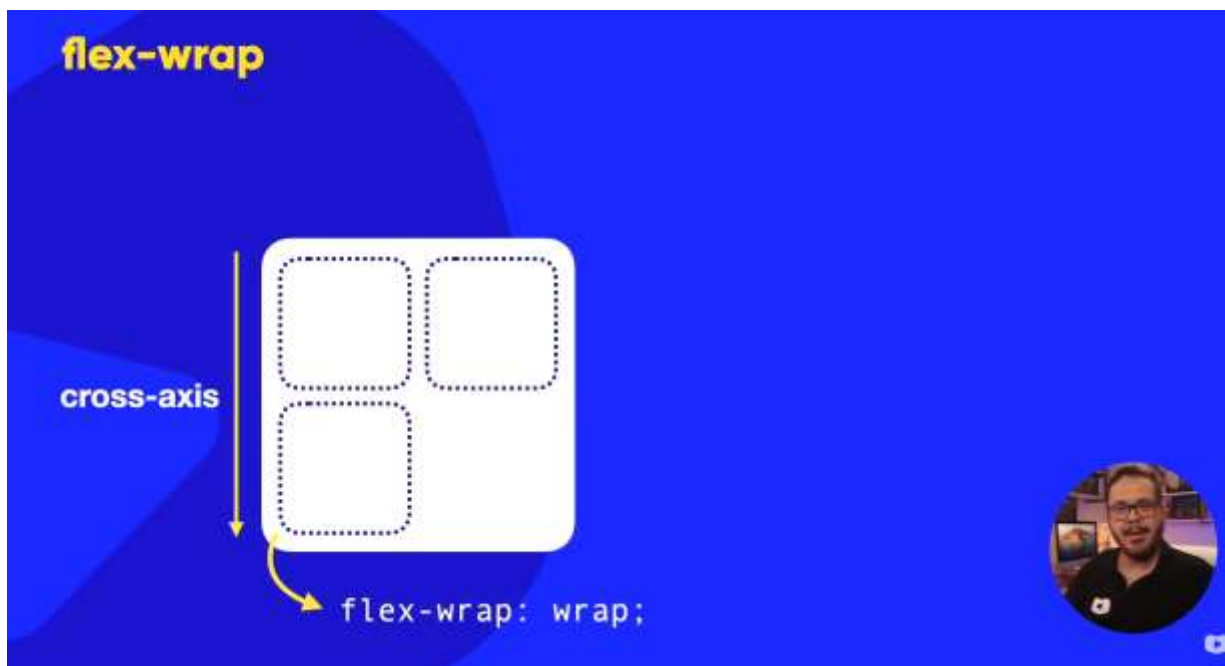
Propriedade: `flex-wrap` / Bloco pai (`Container`)

Por padrão, o Flexbox tenta acomodar todos os itens em uma única linha. Esta propriedade altera essa física.

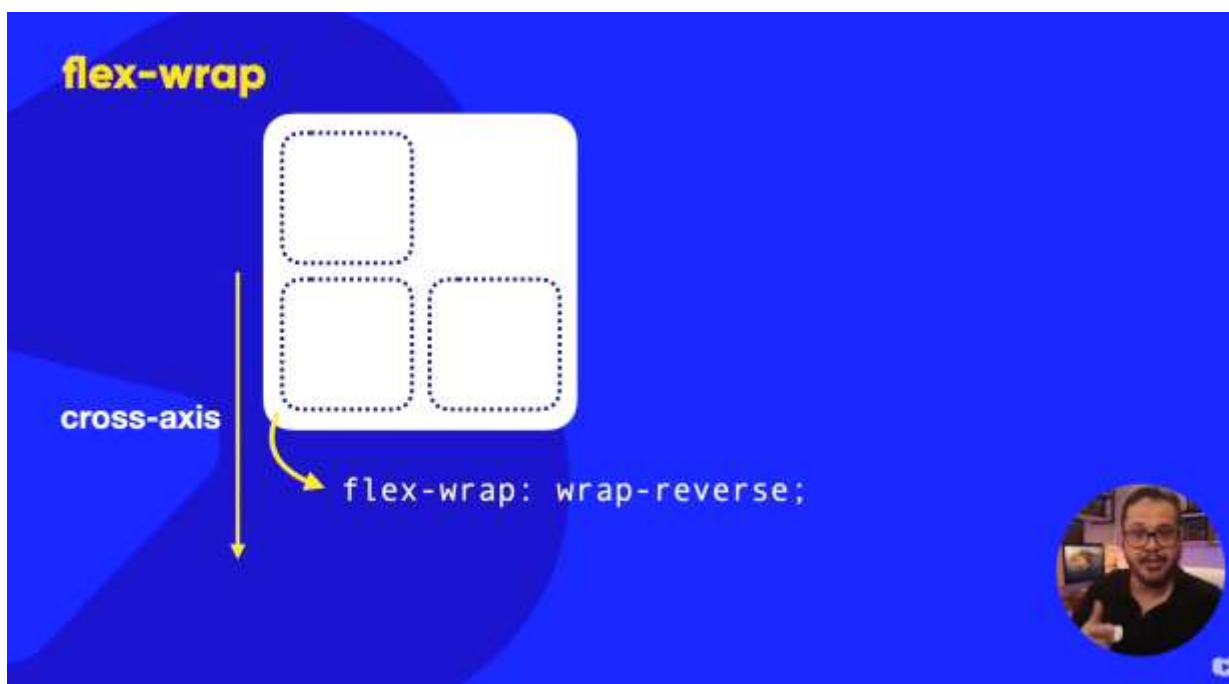
- **nowrap (Padrão):** Força todos os itens em uma única linha, podendo causar *overflow* ou encolhimento excessivo.

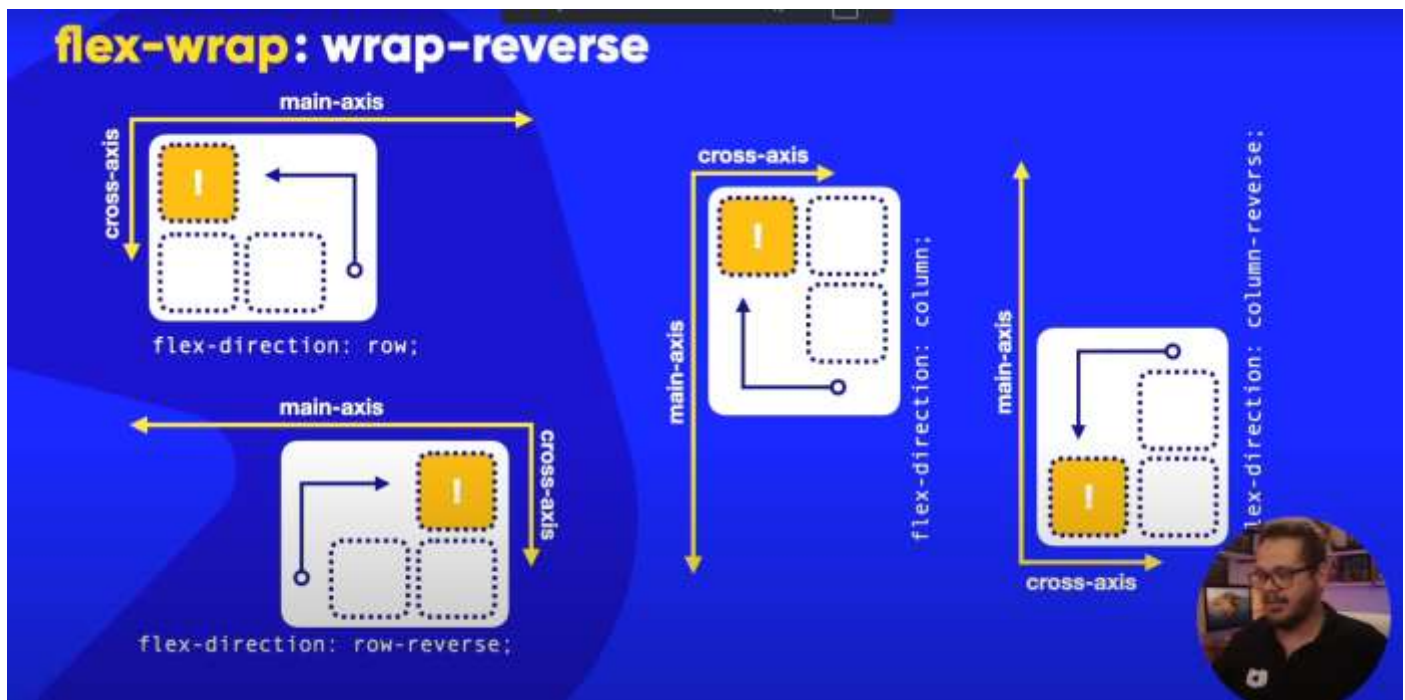


- **wrap:** Permite que os itens quebrem para a próxima linha se não houver espaço suficiente.



- **wrap-reverse:** Quebra linha, mas na ordem inversa (baixo para cima).





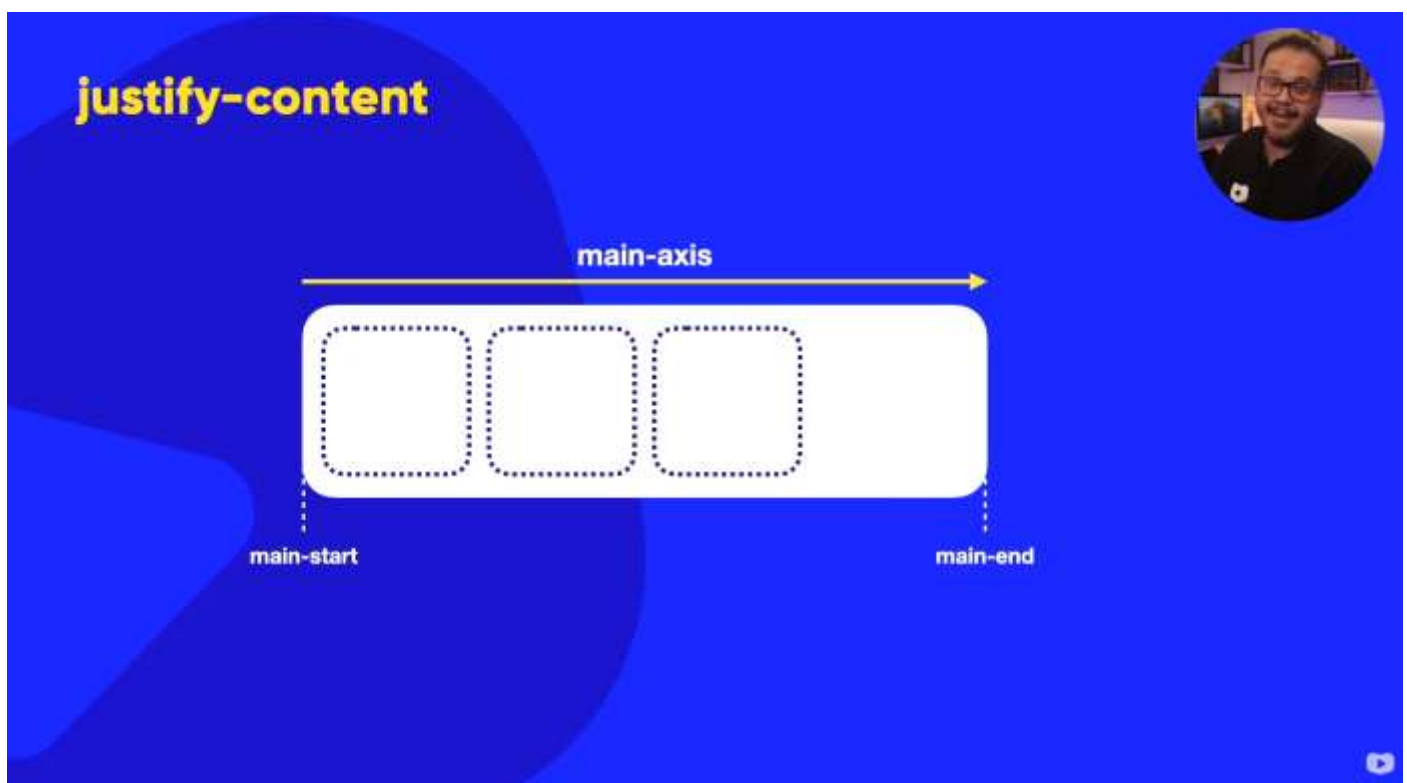
Nota Técnica: A propriedade shorthand **flex-flow** pode ser utilizada para definir direção e quebra simultaneamente:

CSS

```
.container {  
  flex-flow: row wrap; /* Direção linha, permitindo quebra */  
}
```

O encolhimento vai depender dos elementos dentro do container. Os itens no caso

CSS – Propriedade `justify-content` - NO PAI



O que é justify-content?

A propriedade `justify-content` é utilizada em **containers flex** (`display: flex`) para **alinhar os itens flexíveis ao longo do eixo principal (main axis)**.

- Por padrão, o **eixo principal é horizontal** (da esquerda para a direita).
- Ela atua **quando os itens não ocupam todo o espaço disponível** dentro do container.

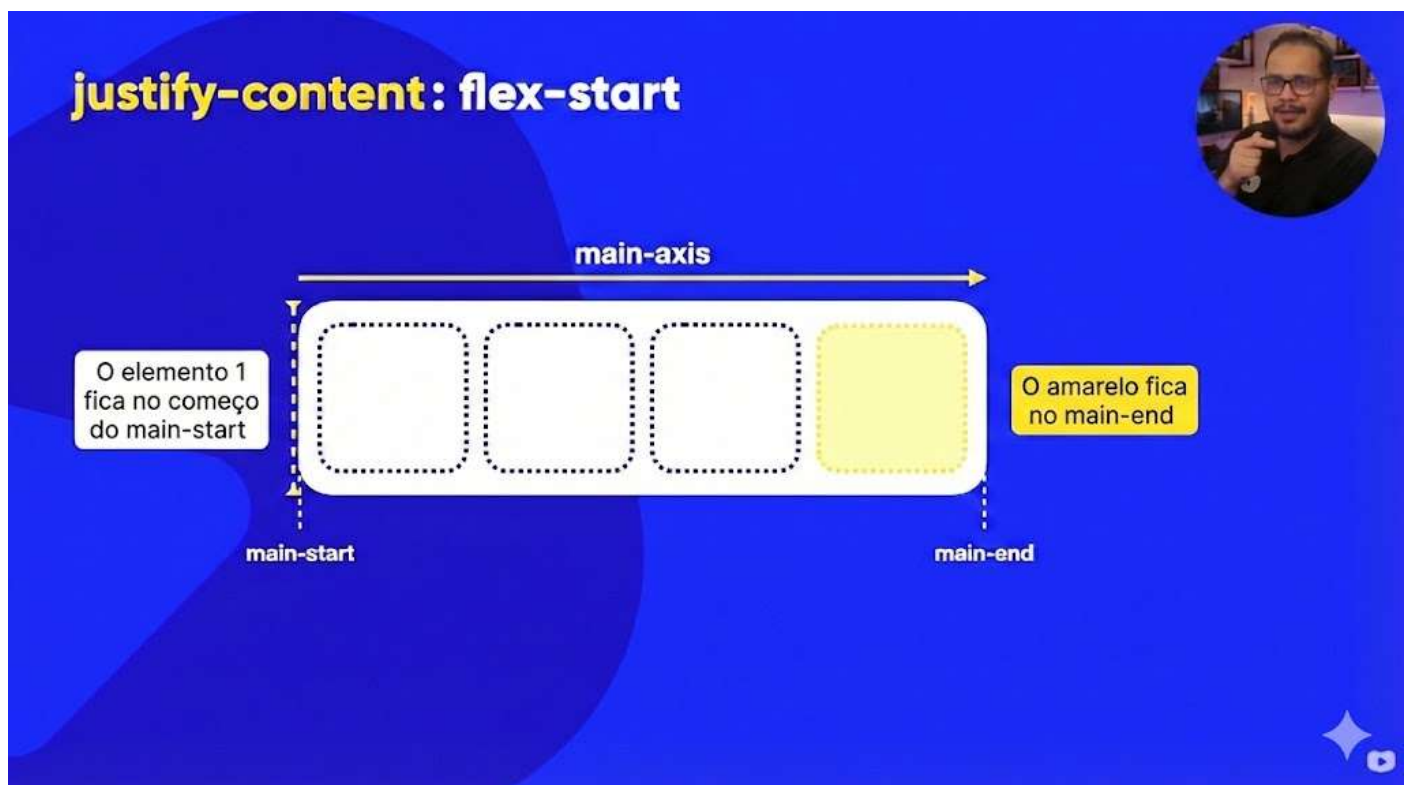
Pré-requisito

Para que `justify-content` funcione, o container precisa ter:

```
display: flex;
```

Valores da propriedade `justify-content`

1. `flex-start` (valor padrão)

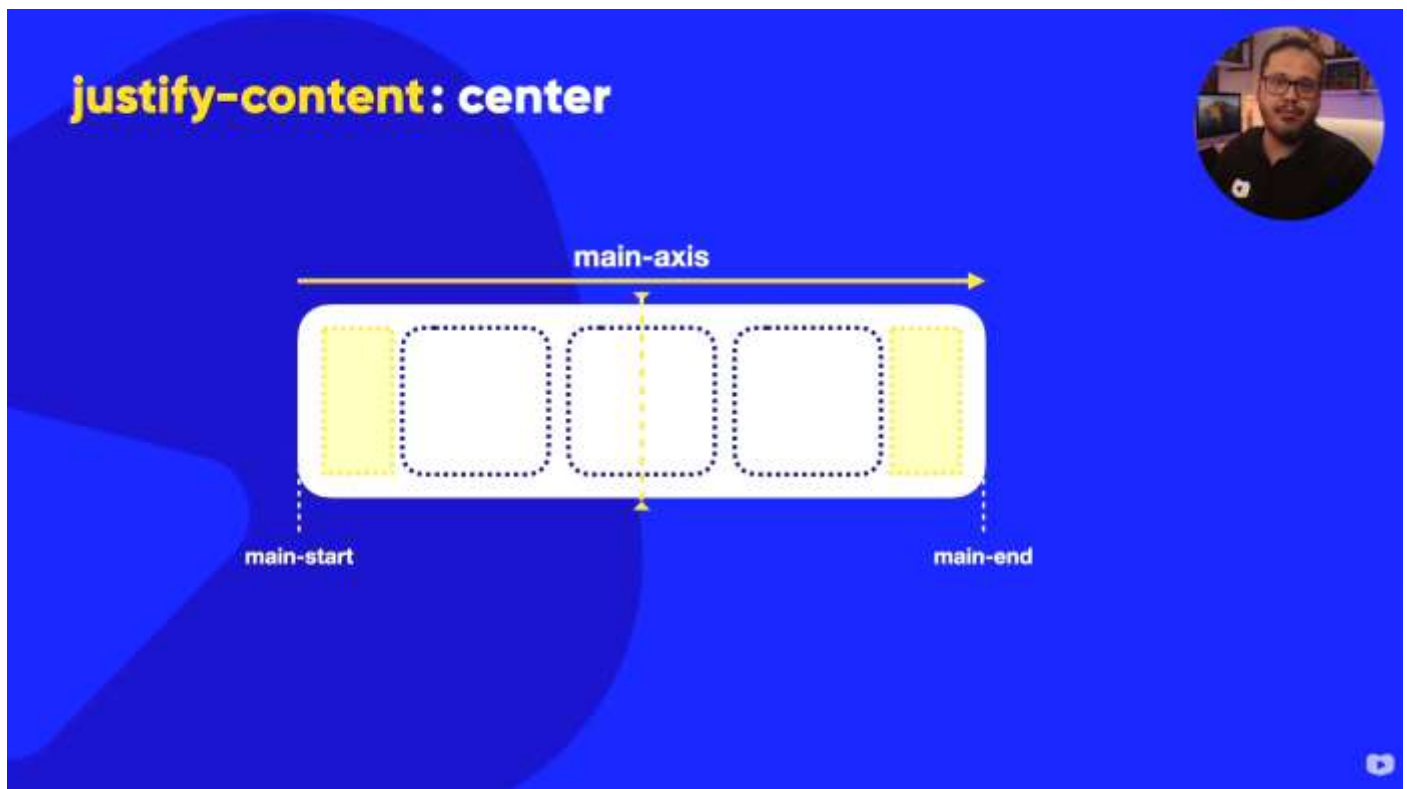


- Alinha os itens **no início do container**, à esquerda.
- Não adiciona espaço entre os itens.

```
.flex-container {  
  display: flex;  
  justify-content: flex-start;  
}
```

Uso comum: layout padrão, menus simples.

2. center

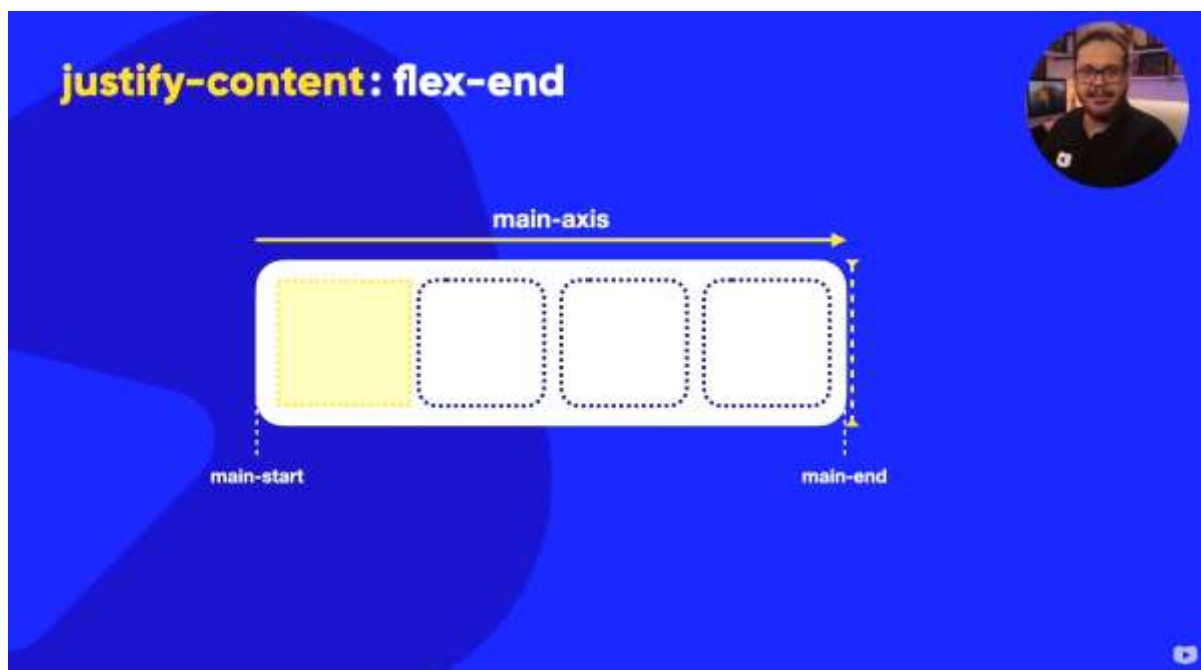


- Centraliza todos os itens **horizontalmente** dentro do container.
- O espaço livre fica igualmente distribuído à esquerda e à direita.

```
.flex-container {  
  display: flex;  
  justify-content: center;  
}
```

Uso comum: centralização de botões, cards ou elementos principais.

3. flex-end

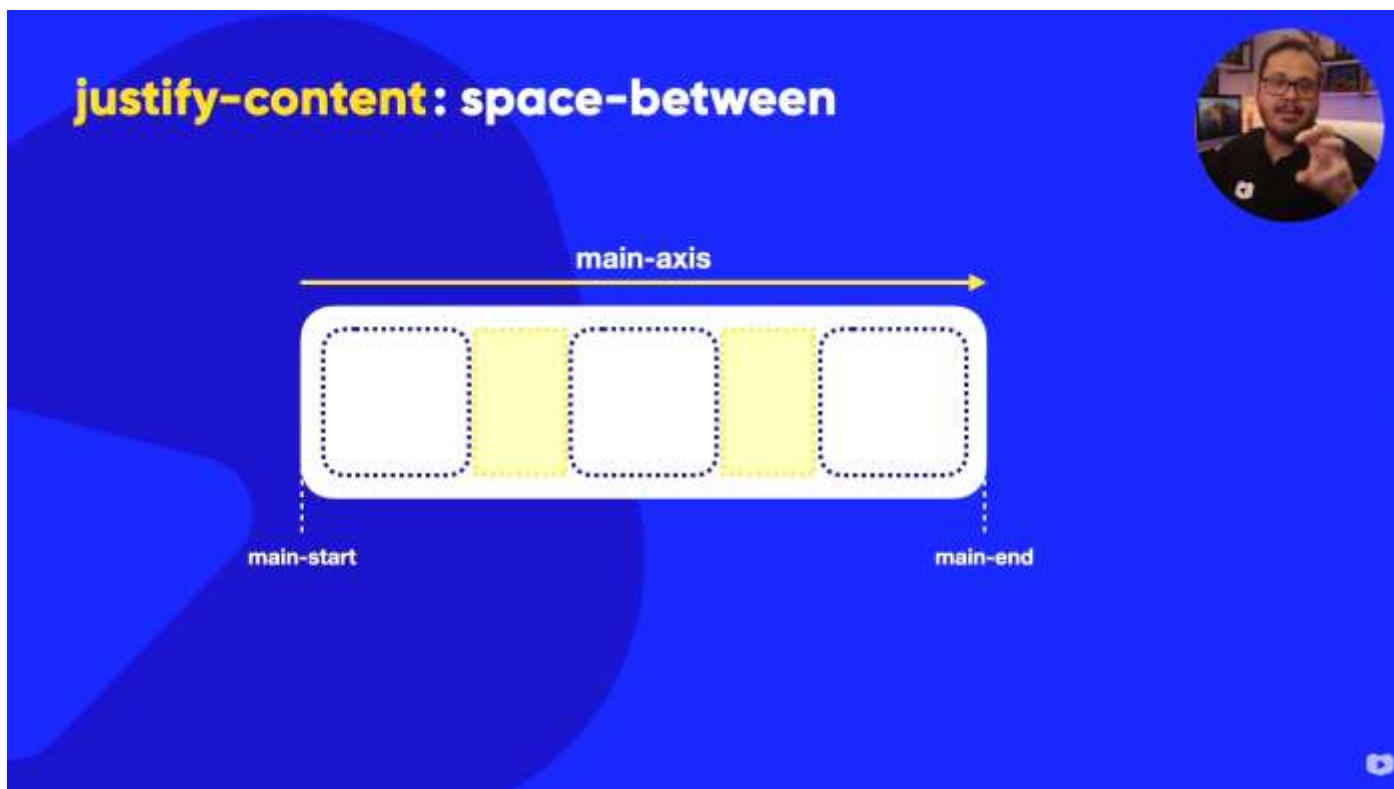


- Alinha os itens **no final do container**, à direita.

```
.flex-container {  
  display: flex;  
  justify-content: flex-end;  
}
```

Uso comum: botões de ação alinhados à direita.

4. space-between

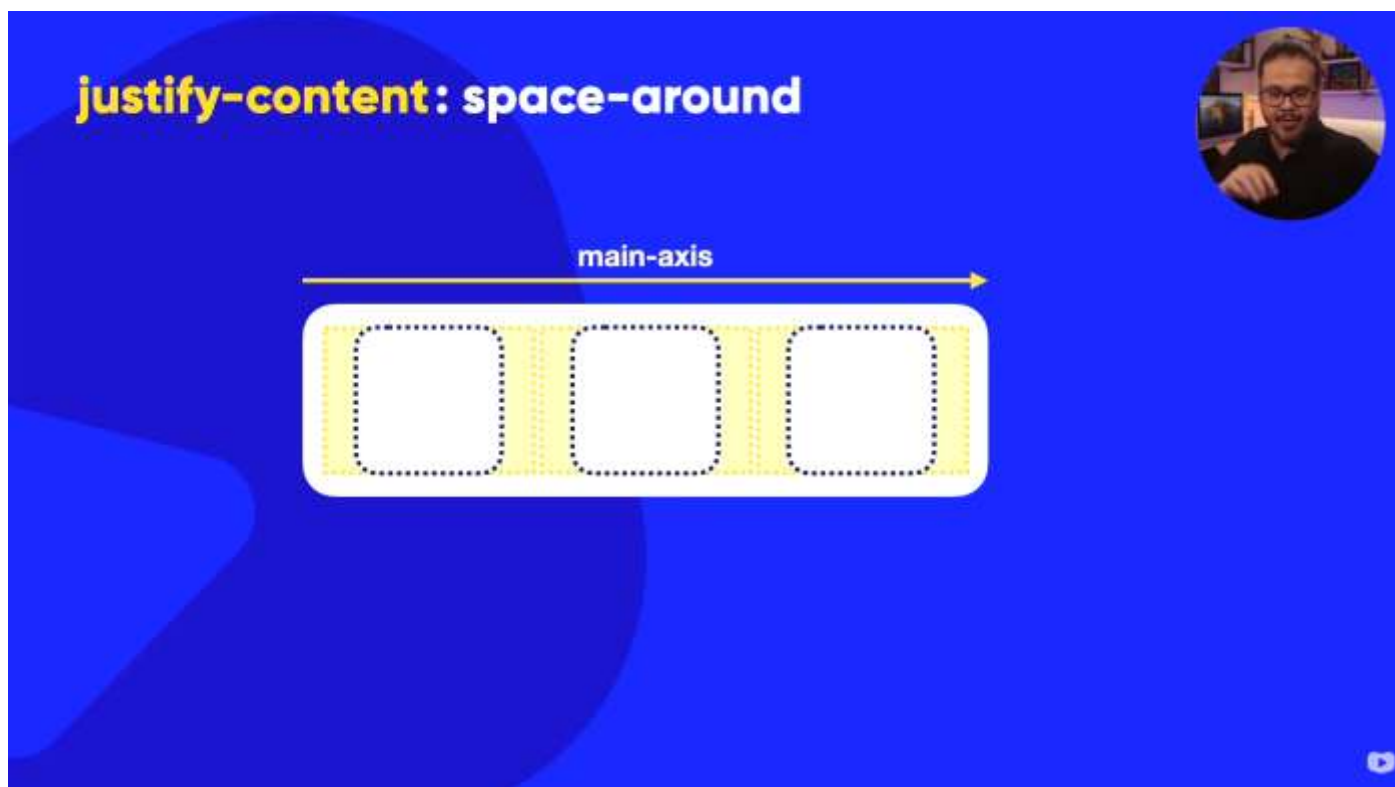


- Distribui os itens com **espaço apenas entre eles**.
- O primeiro item fica colado no main-start e o último no main-end.

```
.flex-container {  
  display: flex;  
  justify-content: space-between;  
}
```

Uso comum: barras de navegação (logo à esquerda e menu à direita).

5. space-around

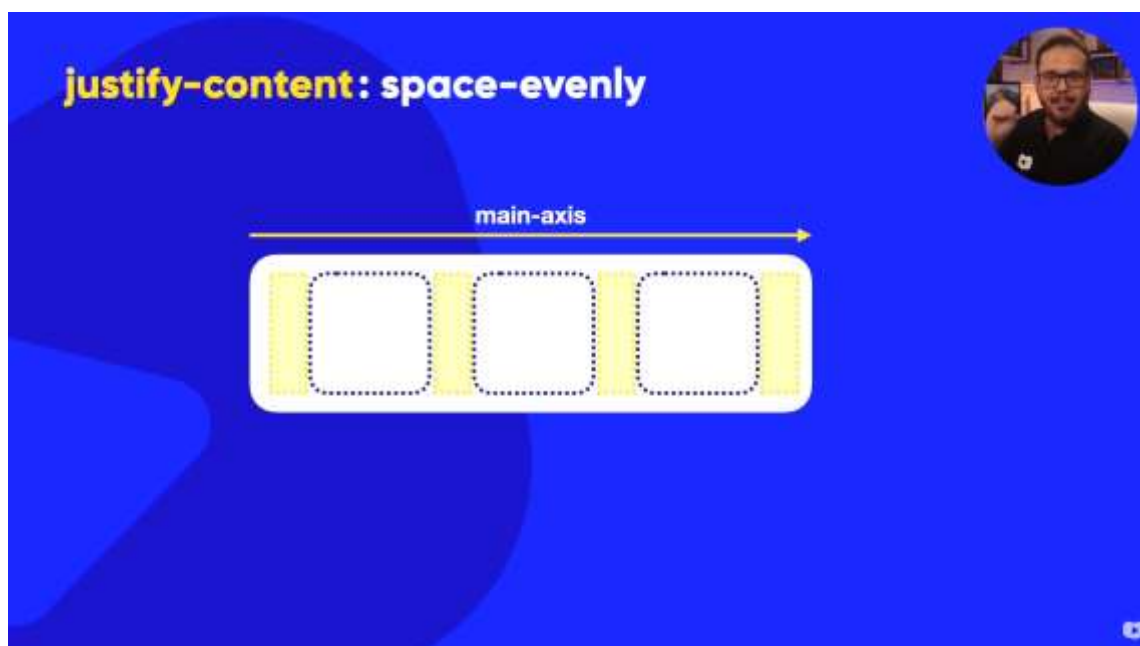


- Distribui os itens com **espaço ao redor de cada item**.
- O espaço entre itens é maior do que nas bordas.
- Ele cria uma caixa os 3 elementos e coloca no meio dessa caixa cada elemento

```
.flex-container {  
  display: flex;  
  justify-content: space-around;  
}
```

Observação: as bordas têm metade do espaço interno entre os itens.

6. space-evenly



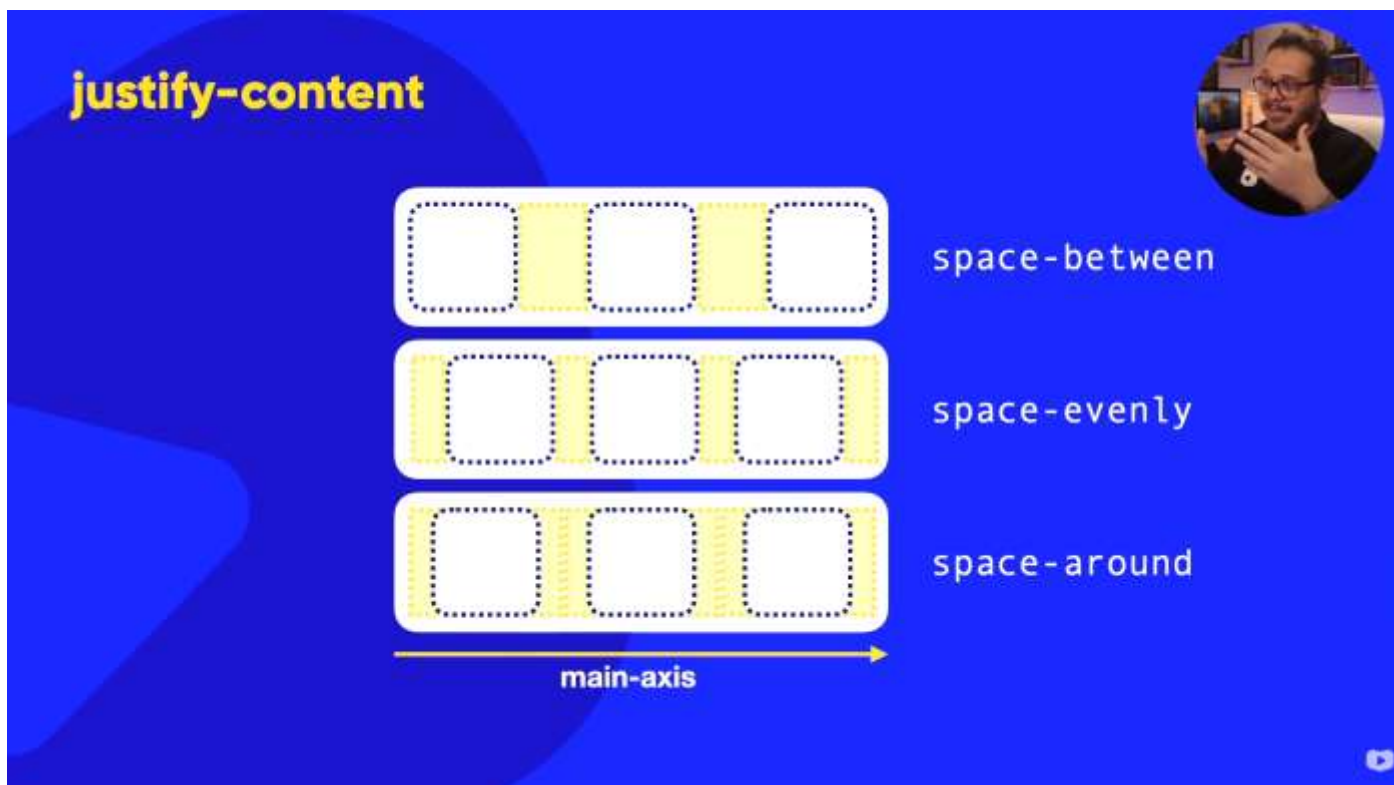
- Distribui os itens com **espaços exatamente iguais**:
 - Entre os itens
 - Nas bordas do container
- Os espaços em amarelo são iguais de tamanho

```
.flex-container {  
  display: flex;  
  justify-content: space-evenly;  
}
```

Uso comum: layouts equilibrados e visualmente simétricos.

Resumo Comparativo

Valor	Distribuição do Espaço
flex-start	Tudo no início
center	Centralizado
flex-end	Tudo no final
space-between	Espaço só entre itens
space-around	Espaço ao redor (bordas menores)
space-evenly	Espaço igual em tudo



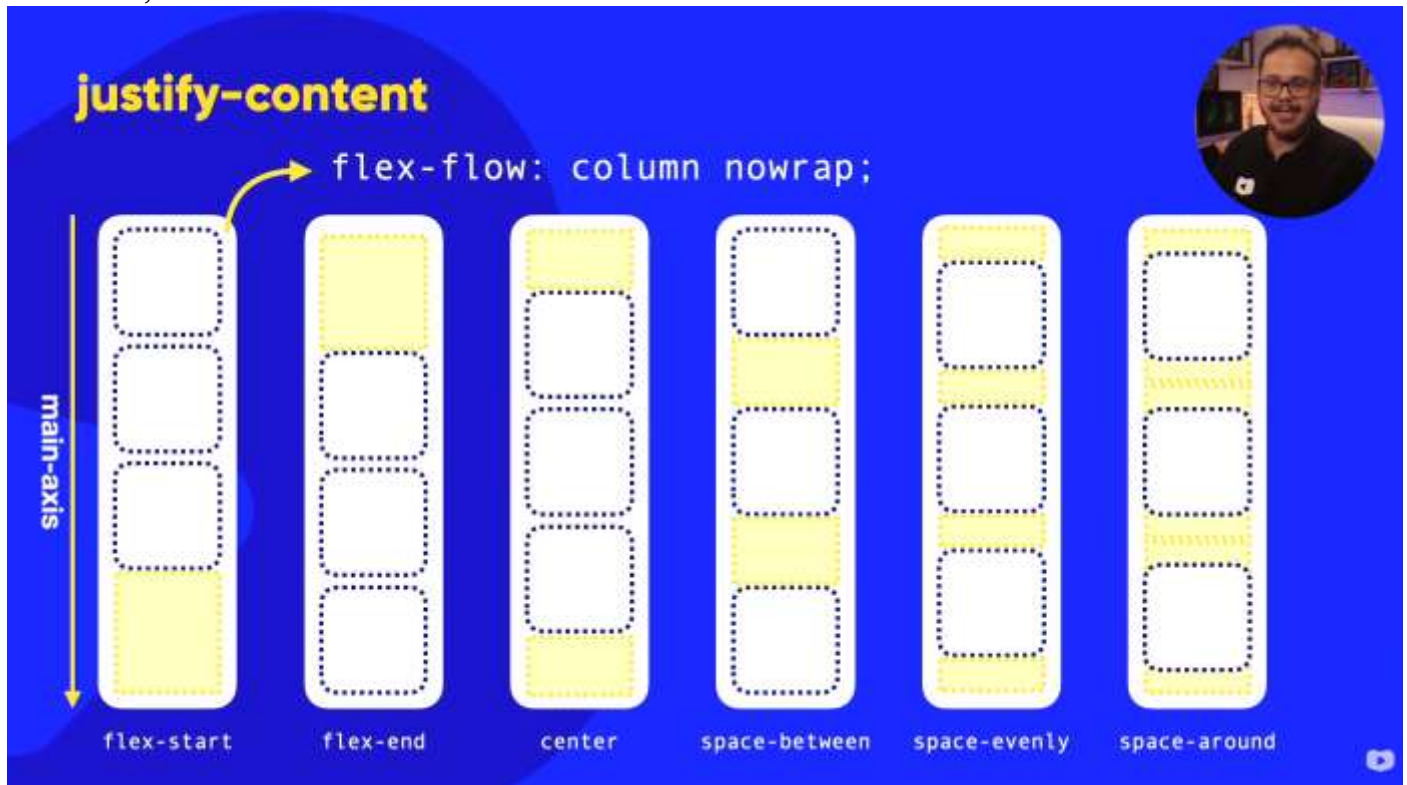
Observação Importante

- `justify-content` atua no **eixo principal**.
- Se o `flex-direction` for alterado para `column`, o eixo principal passa a ser **vertical**, e o comportamento muda.

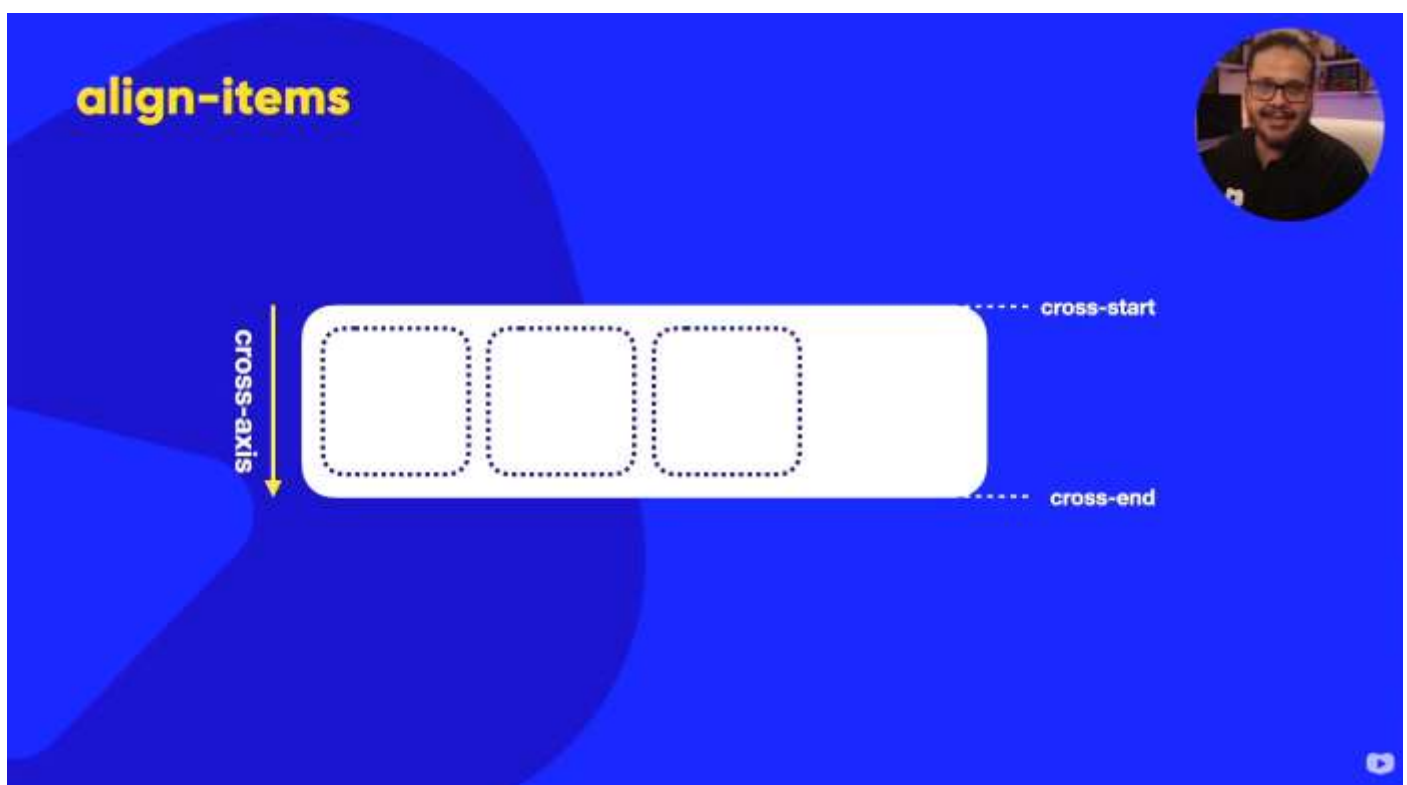
Exemplo:

```
.flex-container {
  display: flex;
  flex-direction: column;
  justify-content: center;
}
```

Neste caso, os itens serão centralizados **verticalmente**.



CSS – Propriedade `align-items` - no pai



O que é `align-items`?

A propriedade `align-items` é usada em **containers flex** (`display: flex`) para **alinhar os itens flexíveis ao longo do eixo transversal (cross axis)**.

- Em um layout padrão (`flex-direction: row`), o **eixo transversal é vertical**.
- Atua quando os itens **não ocupam toda a altura disponível** do container.

Diferença entre os eixos

- **Eixo principal (main axis)** → controlado por `justify-content`
- **Eixo transversal (cross axis)** → controlado por `align-items`

Pré-requisitos

Para usar `align-items`, o container precisa ter:

```
display: flex;
```

Normalmente é necessário definir uma altura para visualizar o efeito:

```
height: 200px;
```

Valores da propriedade `align-items`

2. `stretch` (VALOR PADRÃO)

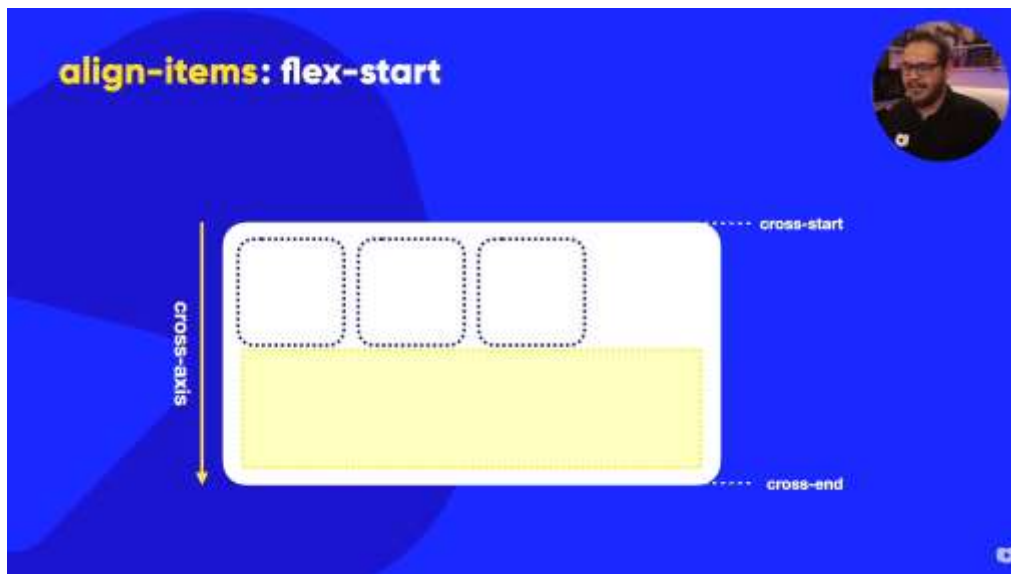


- **Estica os itens** para preencher toda a altura do container.
- Funciona apenas se os itens **não tiverem altura definida**.

```
.flex-container {  
  display: flex;  
  height: 200px;  
  align-items: stretch;  
}
```

Uso comum: colunas com a mesma altura.

3. flex-start



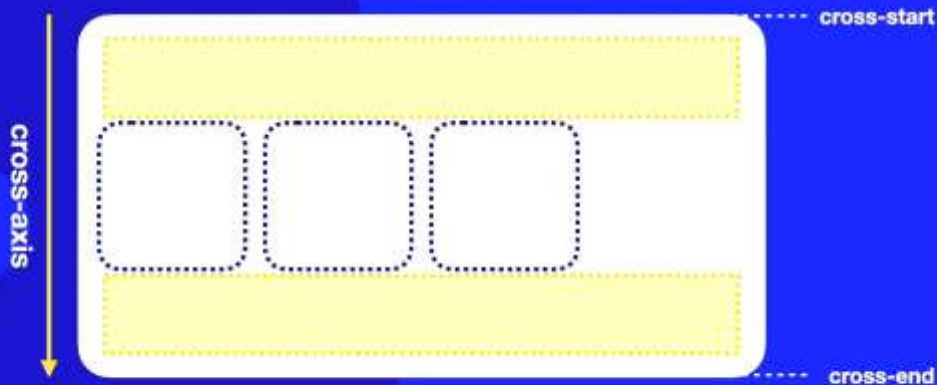
- Alinha os itens **no topo do container**.

```
.flex-container {  
  display: flex;  
  height: 200px;  
  align-items: flex-start;  
}
```

Uso comum: conteúdo alinhado ao topo de cards ou seções.

4. center

align-items: center



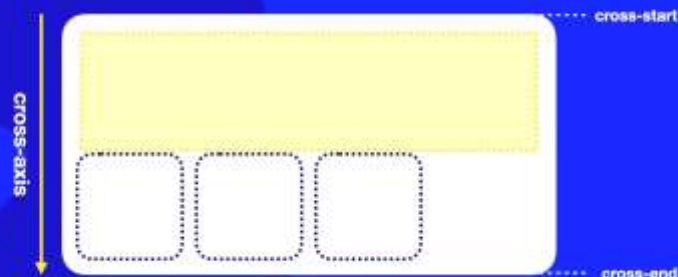
- Centraliza os itens **verticalmente**.

```
.flex-container {  
  display: flex;  
  height: 200px;  
  align-items: center;  
}
```

Uso comum: centralização vertical de elementos (muito comum em layouts modernos).

5. flex-end

align-items: flex-end



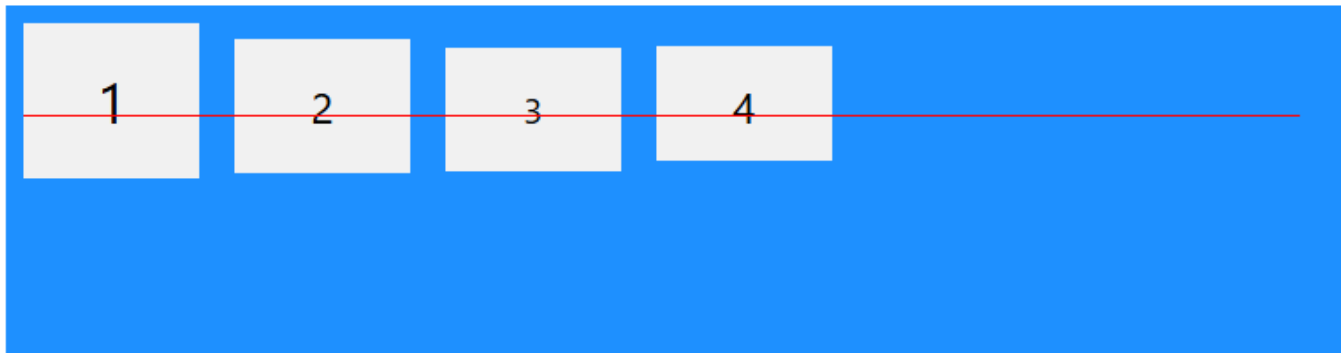
- Alinha os itens **na parte inferior do container**.

```
.flex-container {  
  display: flex;
```

```
height: 200px;
align-items: flex-end;
}
```

Uso comum: rodapés internos ou botões no final de caixas.

7. baseline



- Alinha os itens com base na **linha base do texto (baseline)**.
- Muito útil quando os itens têm **fontes de tamanhos diferentes**.

```
.flex-container {
  display: flex;
  height: 200px;
  align-items: baseline;
}
```

Observação: o alinhamento acontece pelo texto, não pelo tamanho do elemento.

Resumo Comparativo

Valor	Alinhamento Vertical
stretch	Estica os itens (PADRÃO)
flex-start	Topo
center	Centro
flex-end	Base
baseline	Linha do texto

Observação Importante

Se o `flex-direction` for alterado:

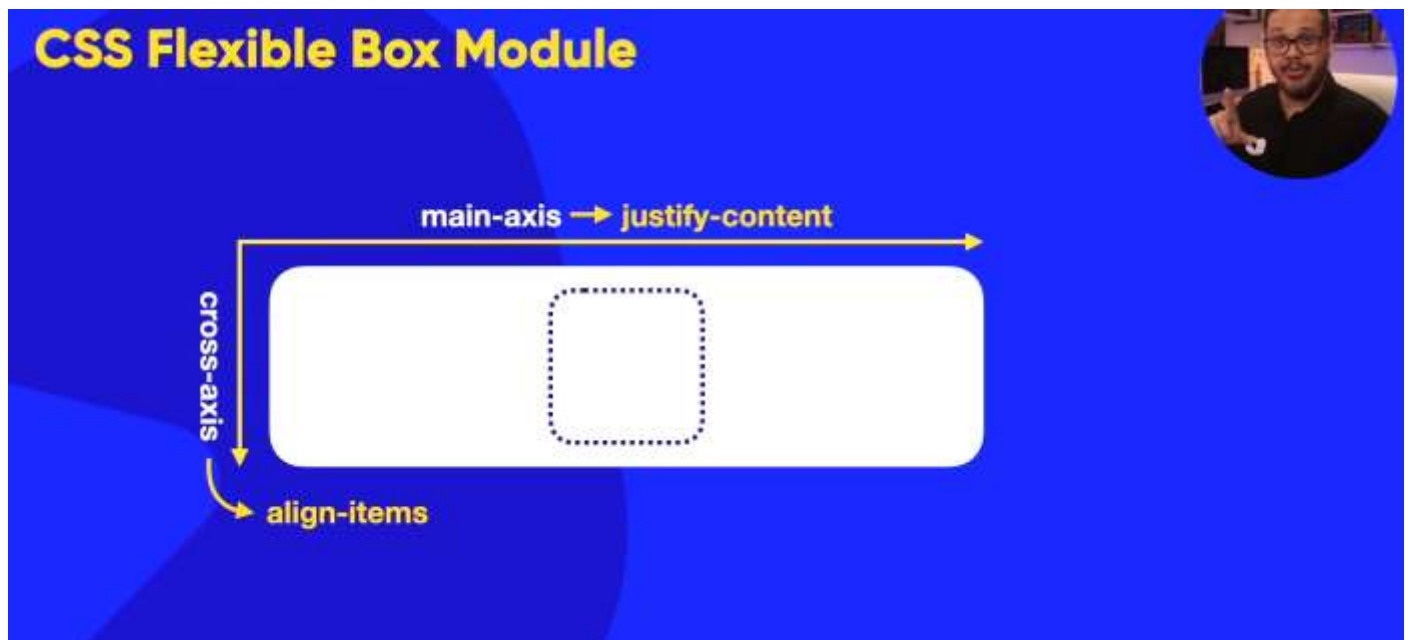
```
flex-direction: column;
```

- O eixo transversal passa a ser **horizontal**.
- Nesse caso, `align-items` alinha os itens **horizontalmente**.

Comparação rápida

`justify-content` → eixo principal
`align-items` → eixo transversal

🎯 Guia Definitivo: Centralização Absoluta em CSS



Centralizar horizontalmente é fácil (`text-align: center` ou `margin: 0 auto`). O desafio real é a **centralização total** (horizontal + vertical). Abaixo estão as 4 melhores técnicas.

1. A Técnica Clássica (Position + Transform)

Ideal para quando você precisa sobrepor elementos ou não está usando um sistema de layout flexível.

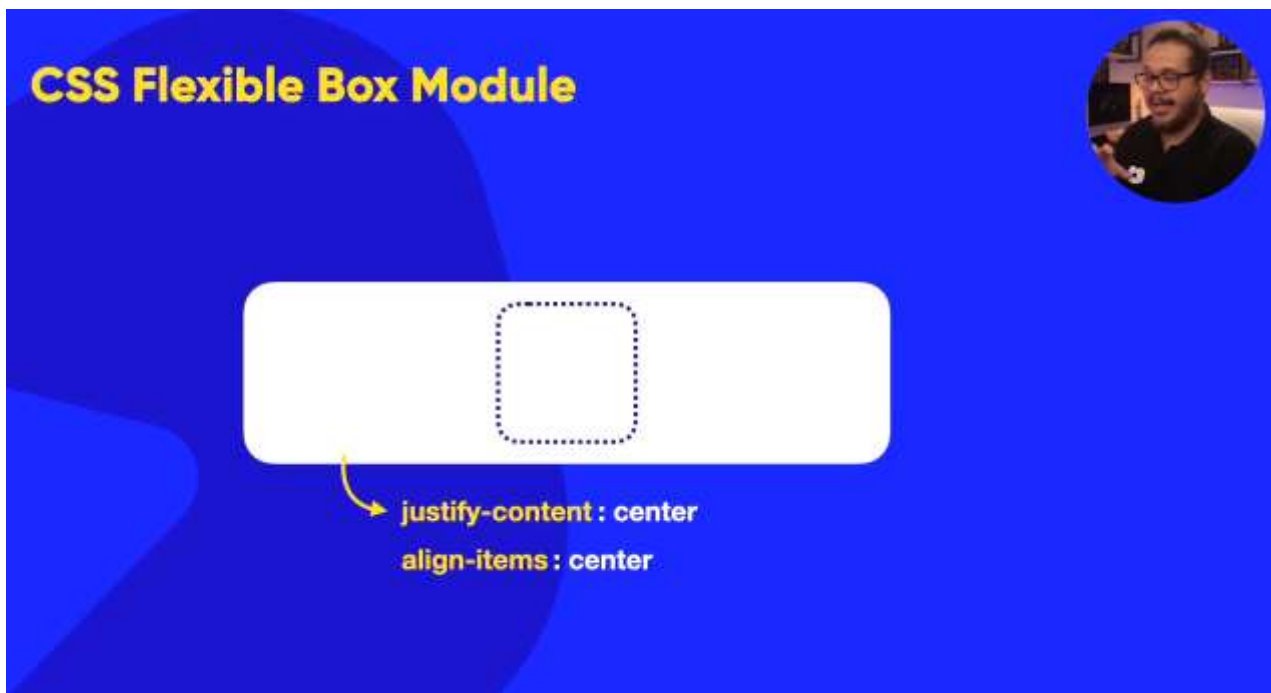
O Conceito:

1. O pai define o contexto (`relative`).
2. O filho se posiciona absolutamente a 50% do topo e da esquerda.
3. O `transform` corrige a posição, puxando o elemento de volta metade do seu próprio tamanho (pois o ponto de referência original é o canto superior esquerdo).

CSS

```
.parent {  
  position: relative;  
}  
  
.child {  
  position: absolute;  
  top: 50%;  
  left: 50%;  
  /* O segredo: move o elemento -50% do seu próprio tamanho */  
  transform: translate(-50%, -50%);  
}
```

2. O Padrão Moderno (Flexbox)

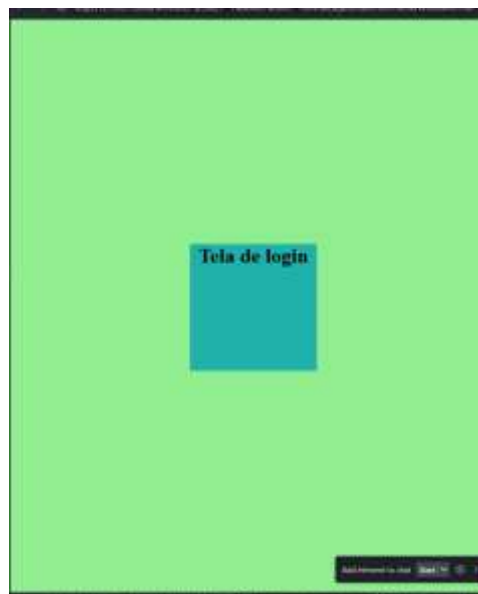


A maneira mais popular e robusta hoje em dia. Transforma o alinhamento em algo intuitivo.

Opção A: Propriedades de Alinhamento

Use `justify-content` para o eixo principal (horizontal) e `align-items` para o eixo transversal (vertical).

```
CSS
.parent {
  display: flex;
  justify-content: center; /* Centraliza na horizontal */
  align-items: center;     /* Centraliza na vertical */
  height: 100vh;          /* Garanta que o pai tenha altura! */
  width: 100vw;           /* Garanta que o pai tenha largura! */
}
```



Opção B: O Truque do Margin Auto

No Flexbox, a margem automática "come" todo o espaço disponível, empurrando o elemento para o centro.

CSS

```
.parent {
  display: flex;
  min-height: 100vh;
}

.child {
  margin: auto; /* Mágica do Flexbox */
}
```

3. O Futuro Robusto (CSS Grid)

O Grid é a ferramenta de layout mais poderosa do CSS. Existem duas formas principais de centralizar com ele.

Opção A: Propriedades Individuais

CSS

```
.parent {
  display: grid;
  height: 100vh;
}

.child {
  justify-self: center; /* Eixo da linha (horizontal) */
  align-self: center; /* Eixo da coluna (vertical) */
}
```

Opção B: A Forma Curta (Recomendada)

Você pode definir tudo no elemento pai com uma única linha: `place-items`.

CSS

```
.parent {
  display: grid;
  height: 100vh;
  place-items: center; /* Atalho para align-items e justify-items */
}
```

4. A Técnica da Viewport (Cenário Específico)

Útil se você quiser centralizar algo relativo à tela inteira sem depender necessariamente do tamanho do pai, usando unidades de viewport (vh).

Nota: Só funciona bem se o elemento deve ficar centralizado na janela de visualização do navegador.

CSS

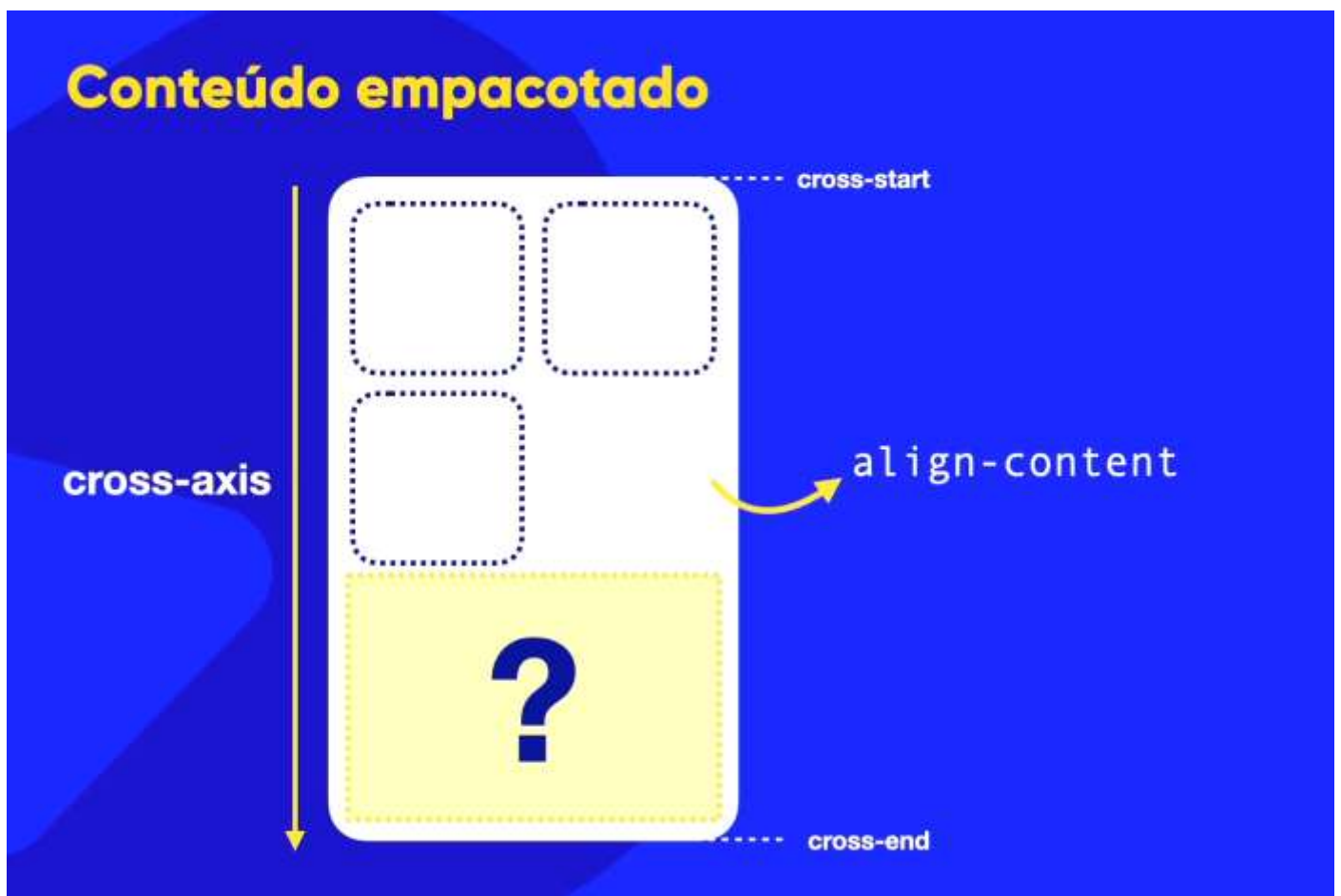
```
.child {
  /* 50vh empurra metade da altura da tela para baixo */
  /* auto centraliza horizontalmente */
  margin: 50vh auto 0;

  /* Corrige a altura do próprio elemento */
  transform: translateY(-50%);
}
```

Técnica	Quando usar?	Prós	Contras
Flexbox	90% dos casos (Menus, Botões, Cards).	Sintaxe moderna, muito suporte.	Requer <code>display: flex</code> no pai.
Grid	Layouts complexos ou centralização "brutal".	<code>place-items: center</code> é imbatível.	Pode ser excessivo para coisas simples.
Position	Overlays, Modais, Pop-ups.	Não afeta outros elementos no fluxo.	Requer mais código e <code>position: relative</code> no pai.
Margin Auto	Dentro de Flexbox.	O código mais limpo possível.	Só funciona se o pai for Flex.

Alinhamento de elementos empacotados no Flexbox

Align-content



1. O Conceito Principal

A propriedade `align-content` controla **como as linhas** de um container flexível (Flexbox) ou de um Grid são distribuídas ao longo do **Eixo Transversal** (Cross Axis).

Analogia: Imagine um parágrafo de texto no Word. O `align-content` é como o alinhamento vertical das linhas desse parágrafo dentro da página (se elas ficam todas no topo, no meio, ou espalhadas).

2. A Regra de Ouro (MUITO IMPORTANTE) ⚠

Para o `align-content` funcionar no Flexbox, **duas condições** são obrigatórias:

1. O container deve ter `display: flex`.
2. O container deve permitir quebra de linha (`flex-wrap: wrap`).

Por quê? Se o conteúdo não quebrar linha (ou se houver apenas uma linha de itens), essa propriedade **não** fará nada.

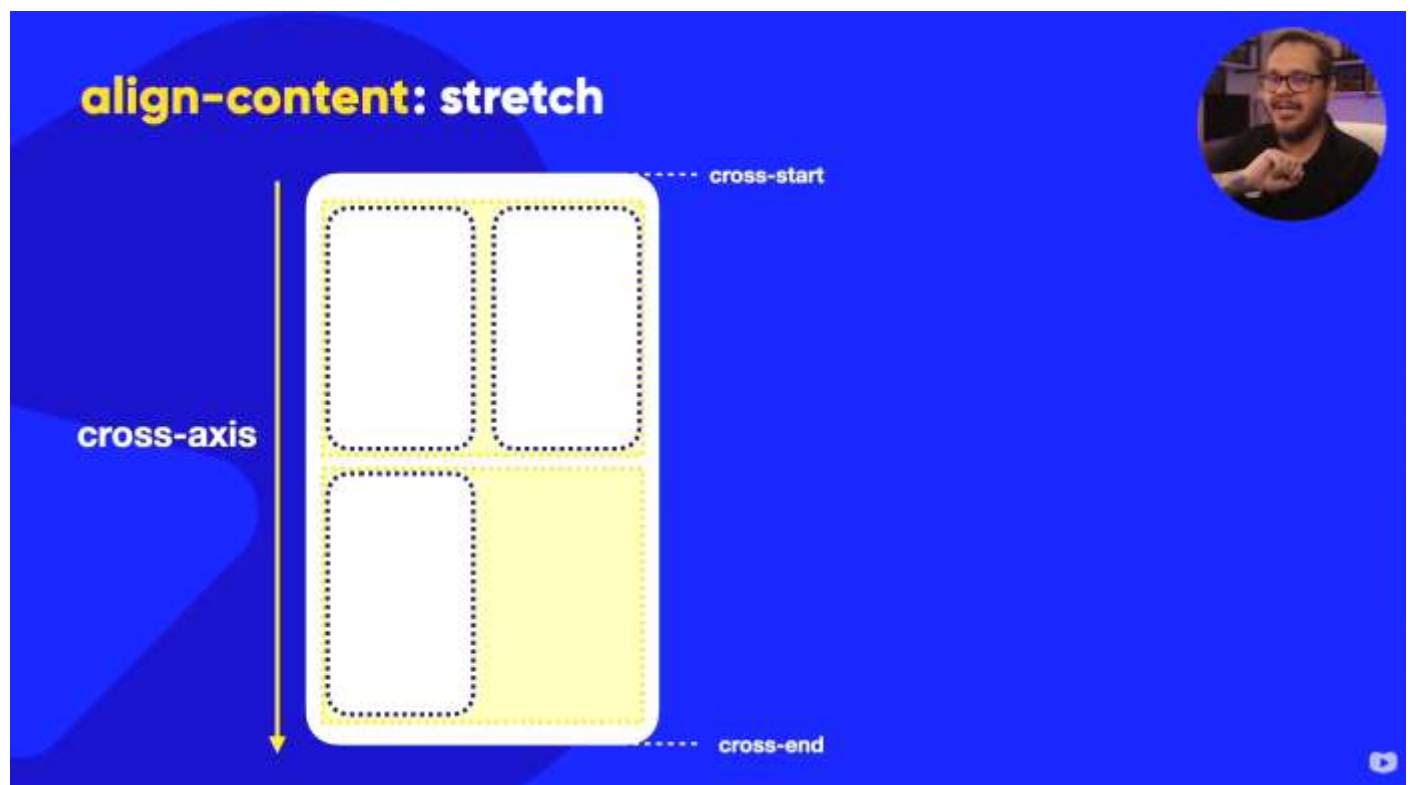
3. Sintaxe e Valores

CSS

```
.container {  
  display: flex;  
  flex-wrap: wrap; /* Obrigatório */  
  height: 600px; /* Precisa de altura sobrando para ver o efeito */  
  align-content: valor;  
}
```

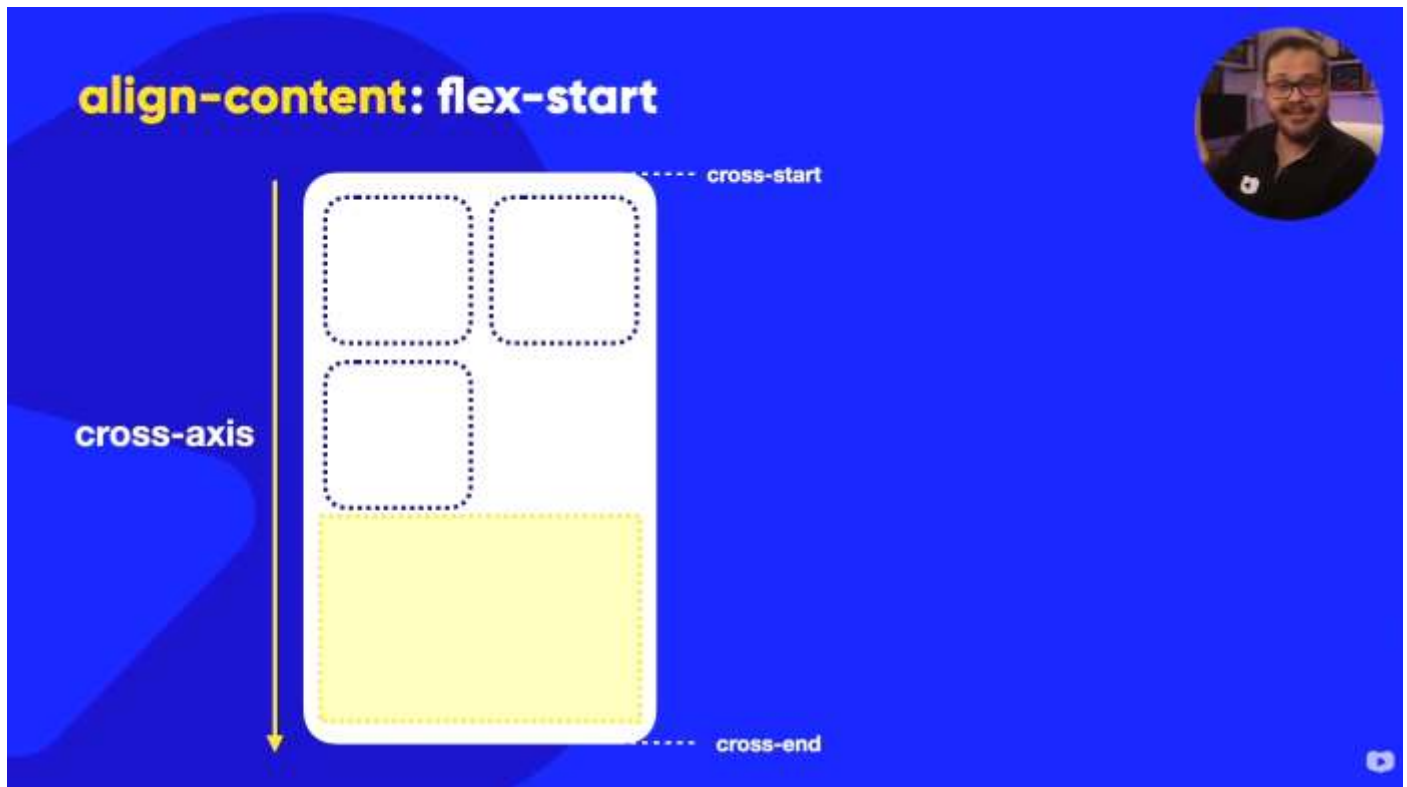
Stretch (PADRÃO)

(Padrão) As linhas se esticam para ocupar todo o espaço disponível. Ficam “gordinhas”.



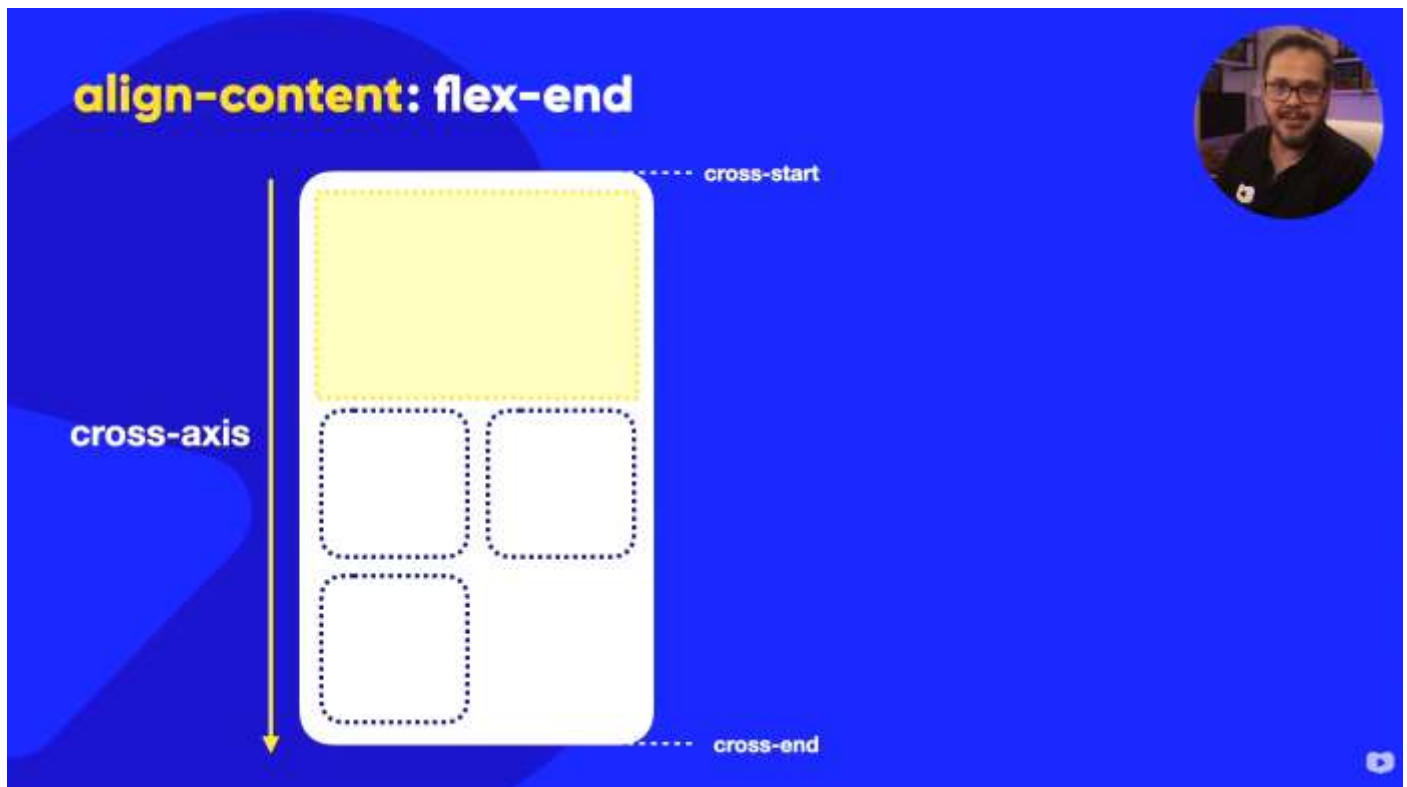
flex-start

Todas as linhas são empacotadas no início do container (no cross-start).



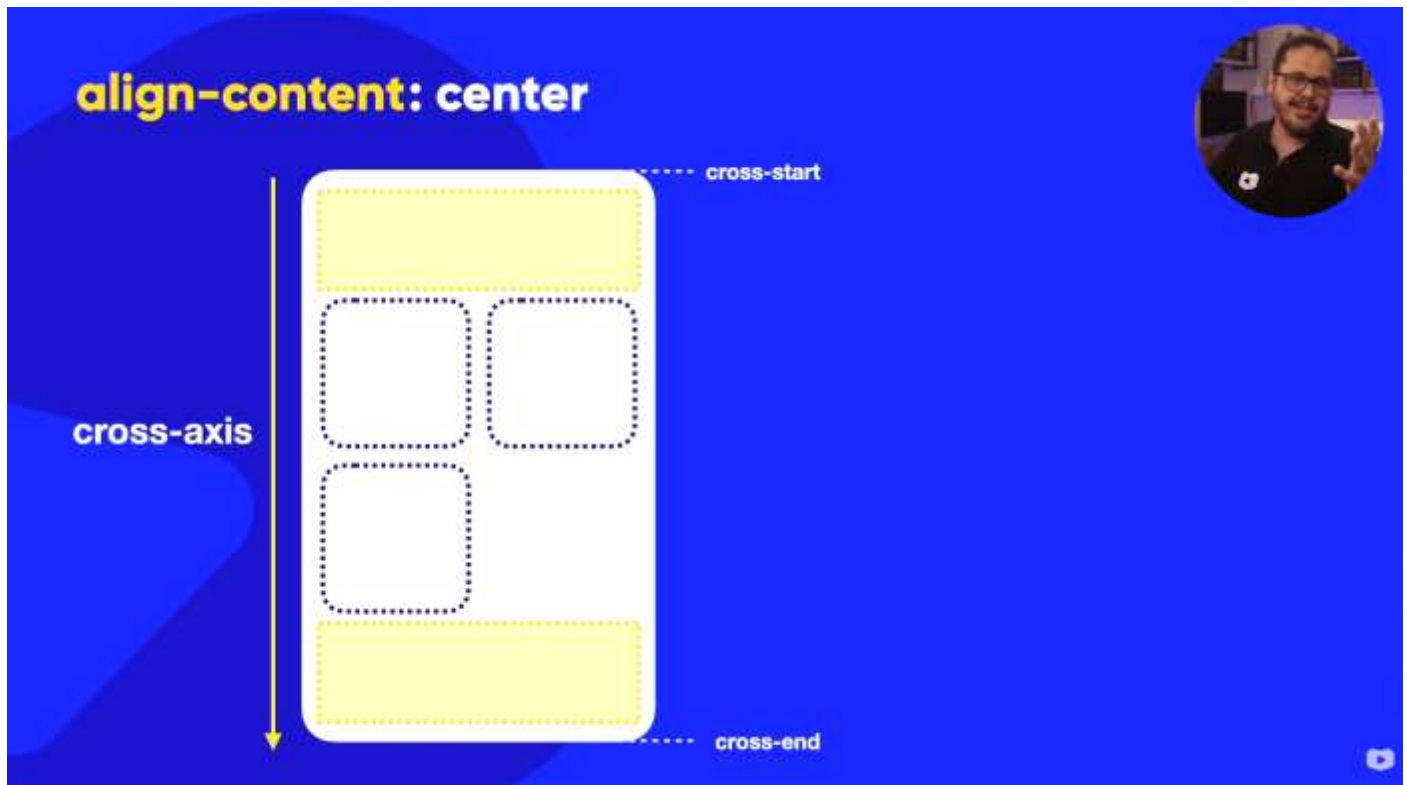
flex-end

Todas as linhas são empacotadas no final do container (cross-end).



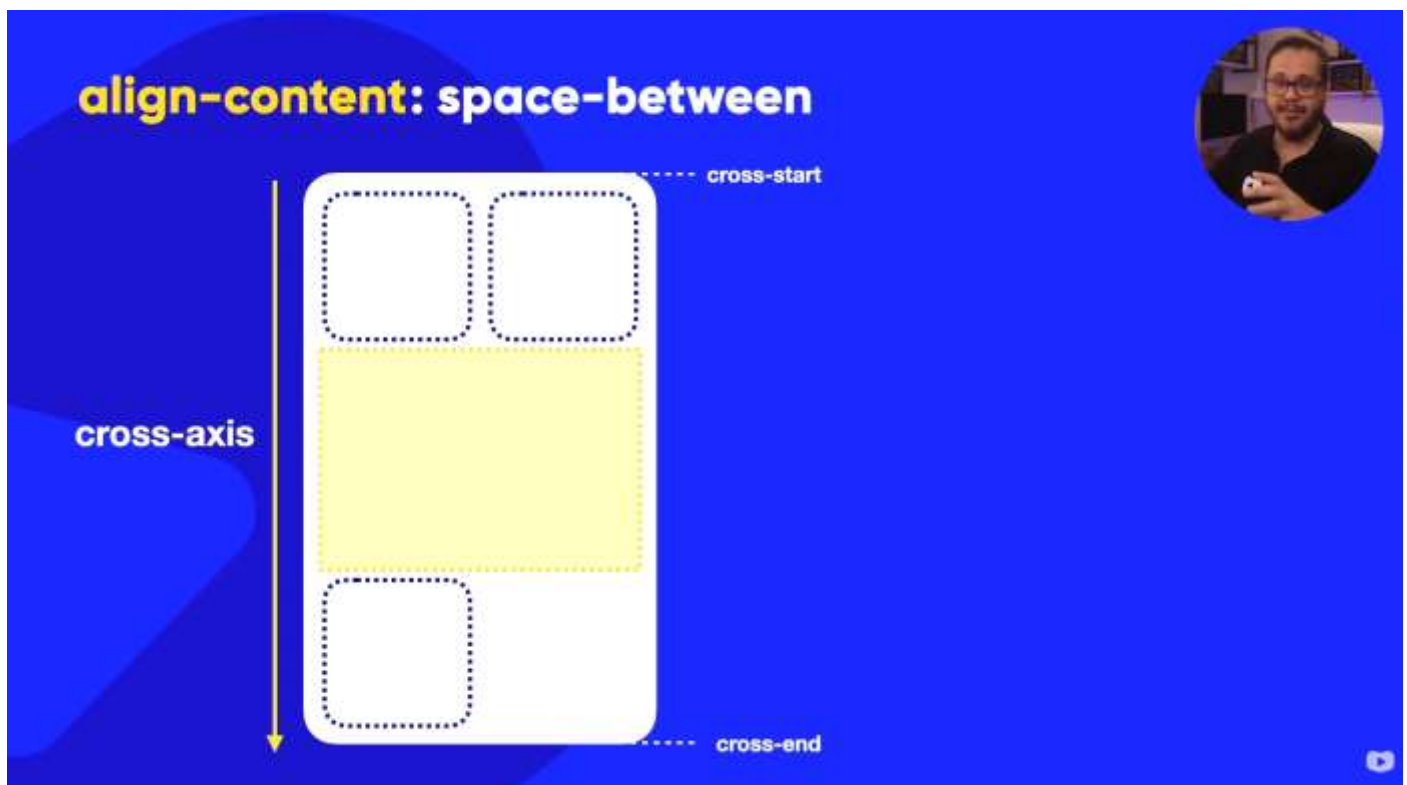
center

Todas as linhas ficam agrupadas no centro do container.



space-between

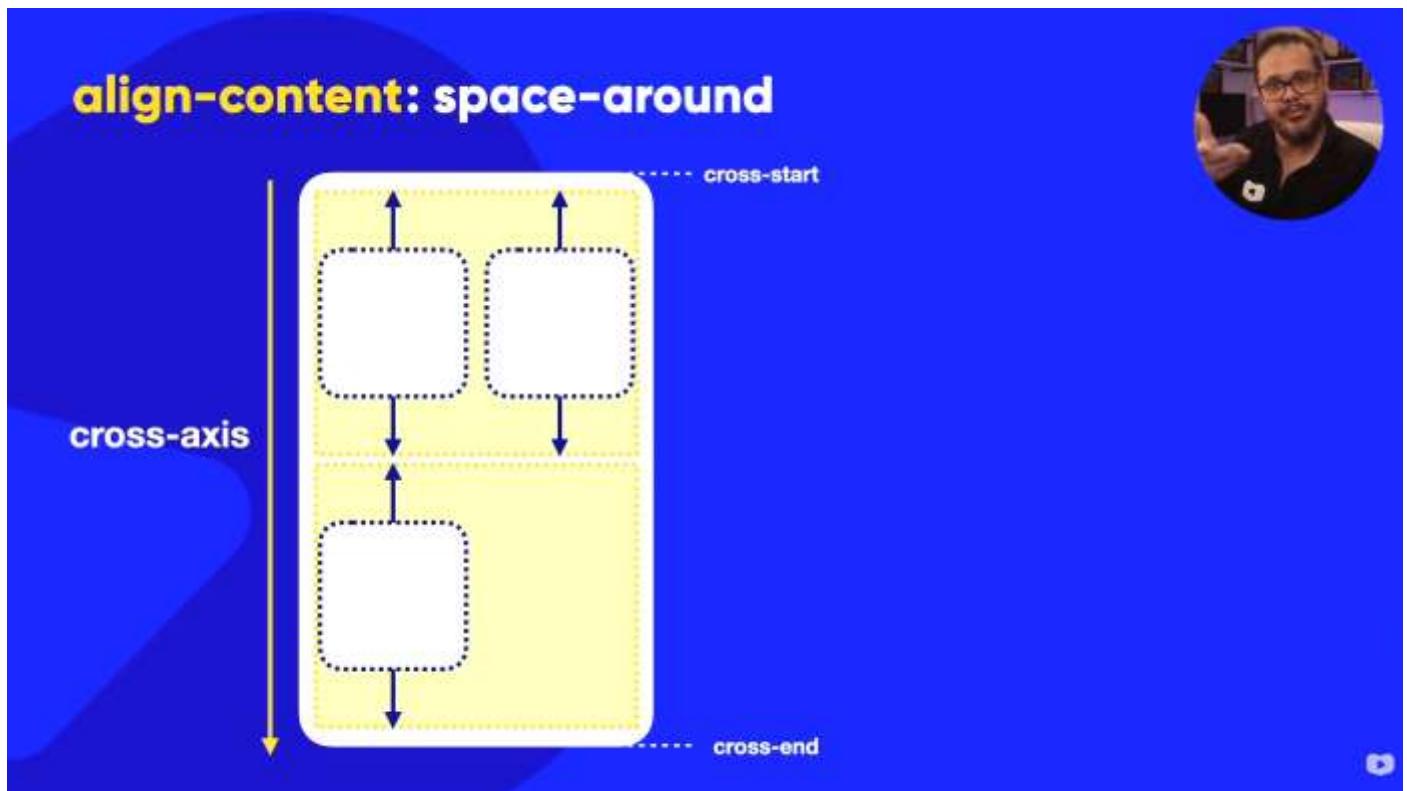
A primeira linha cola no topo, a última cola no fundo, e o espaço sobra apenas entre as linhas.



space-around

Espaço igual ao redor de cada linha.

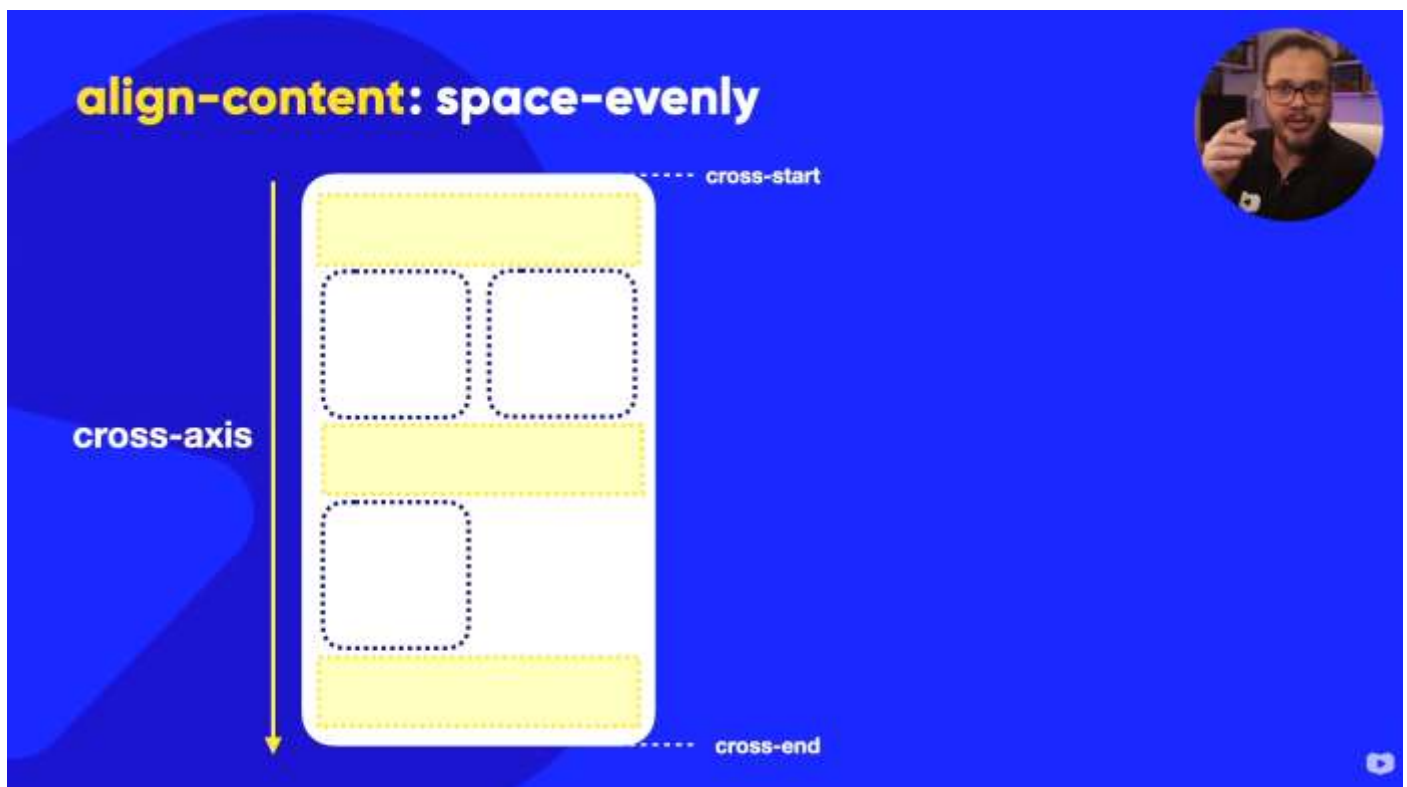
👉 O espaço entre as linhas é o dobro do espaço nas bordas.



space-evenly

Distribui o espaço vazio de forma perfeitamente igual.

👉 Topo, espaço entre linhas e fundo têm a mesma medida.



Diferença crucial: `justify-content` × `align-items` × `align-content`

Antes de tudo, lembre:

- `justify-` → **eixo principal (main-axis)**
 - `align-` → **eixo transversal (cross-axis)**
-

`justify-content`

- Alinha os itens **ao longo do eixo principal**.
- Funciona **em uma ou várias linhas**.
- Controla o **espaçamento horizontal** (row) ou **vertical** (column).

Exemplos:

`flex-start`, `center`, `flex-end`, `space-between`, `space-around`, `space-evenly`

 Use quando quiser controlar **como os itens se distribuem no eixo principal**.

`align-items`

- Alinha os **itens dentro de cada linha**, no eixo cruzado.
- Atua **item por item**.
- Funciona mesmo com **uma única linha**.

Exemplos:

`stretch` (padrão), `center`, `flex-start`, `flex-end`, `baseline`

 Use quando quiser alinhar **os elementos dentro da linha**.

`align-content`

- Alinha o **conjunto de linhas** no eixo cruzado.
-  **Só funciona com múltiplas linhas** (`flex-wrap: wrap`).
- Controla o **espaçamento entre linhas**.

Exemplos:

`flex-start`, `center`, `space-between`, `space-around`, `space-evenly`, `stretch`

 Use quando quiser alinhar **as linhas entre si**.



Regra de ouro (decora isso):

Situação	Propriedade
Espaçamento no eixo principal	<code>justify-content</code>
Alinhar itens dentro da linha	<code>align-items</code>
Espaçamento entre linhas	<code>align-content</code>



Frase clássica de entrevista:

`justify-content` distribui os itens no eixo principal;
`align-items` alinha os itens na linha;
`align-content` alinha as linhas no container.

5. Uso no CSS Grid

O `align-content` também funciona no CSS Grid Layout. A lógica é a mesma, mas alinha as trilhas (tracks) da grade ao longo do eixo de bloco (vertical).

CSS

```
.grid-container {  
  display: grid;  
  align-content: end; /* Joga toda a grade para o final do container */  
}
```

6. Exemplo Prático para o Caderno

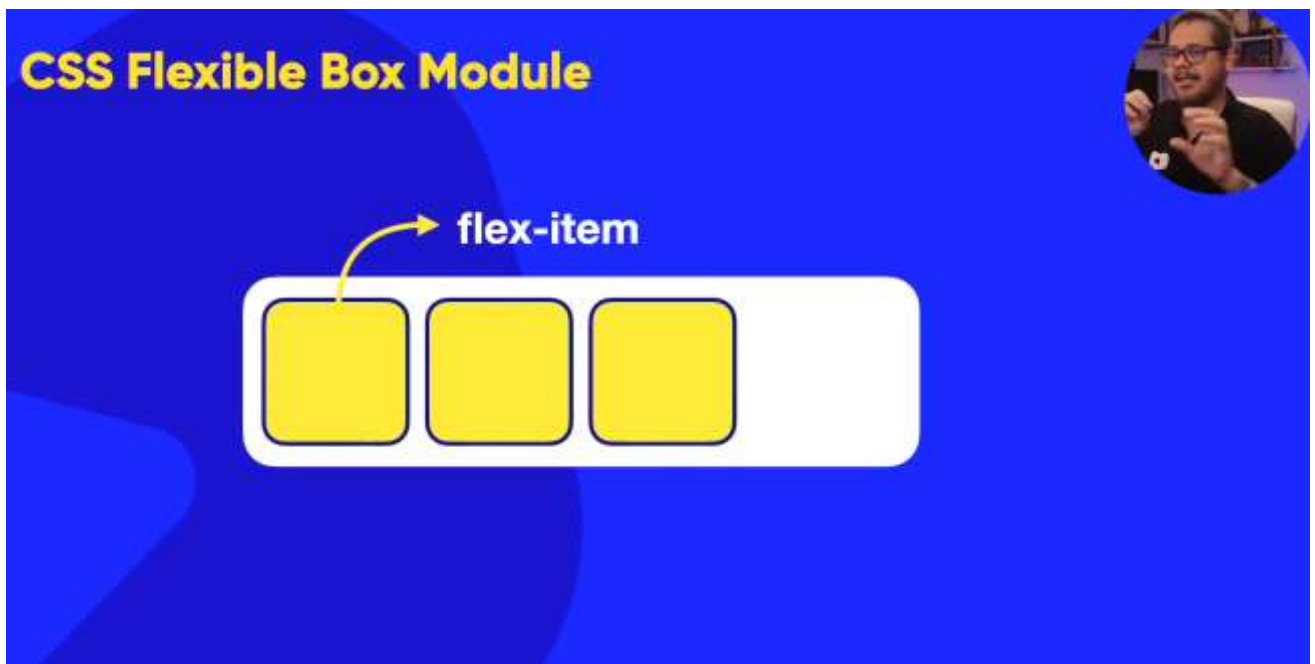
Desenhe ou anote este cenário para fixar:

Cenário: Você tem uma caixa (`div`) de 500px de altura. Dentro dela, vários botões que ocupam 3 linhas.

1. **Sem `align-content` (`stretch`):** As 3 linhas se esticam e ocupam os 500px inteiros.
 2. **Com `align-content: center`:** As 3 linhas se juntam no meio, deixando espaço vazio no topo e no fundo.
 3. **Com `align-content: space-between`:** Uma linha no topo, uma no meio, uma no fundo.
-

🧠 CSS Property `order`

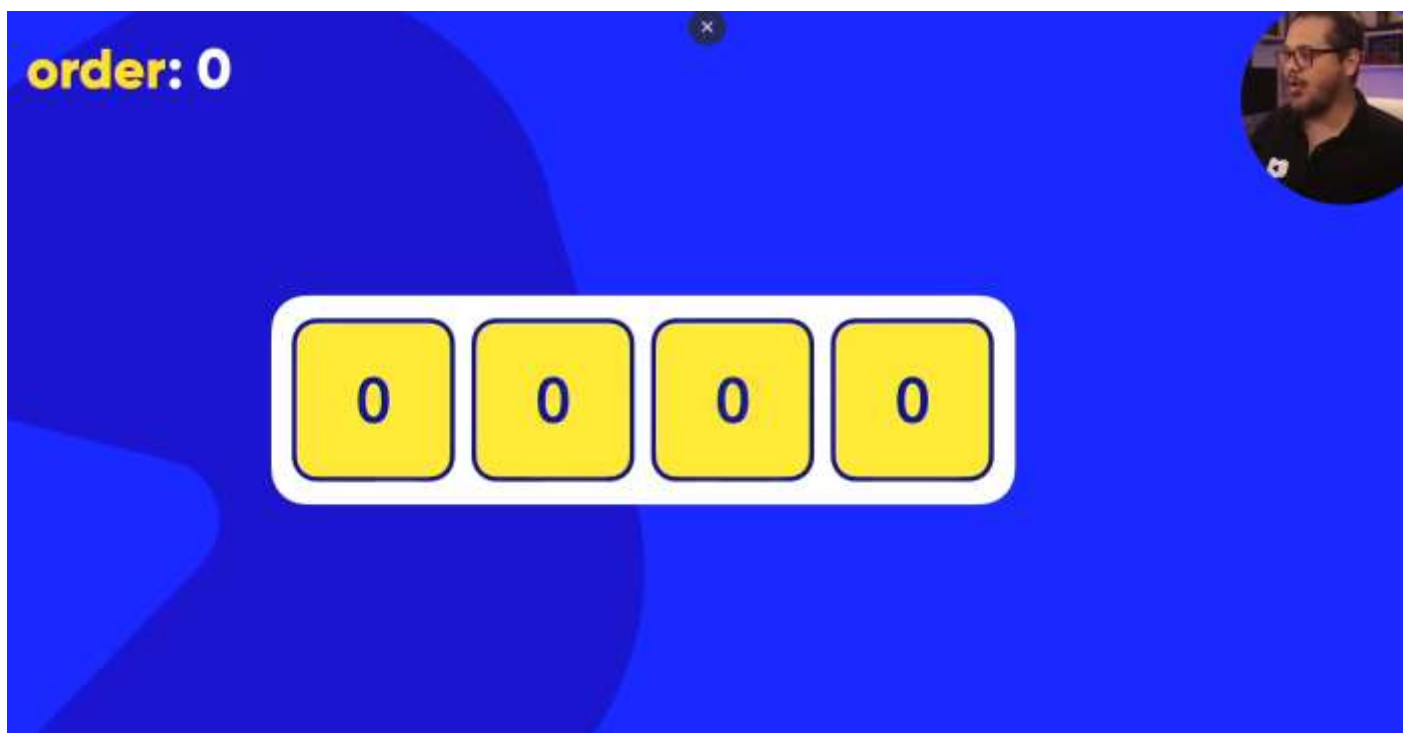
1. O que é?



A propriedade `order` altera a **ordem visual** dos elementos, sem mudar a ordem deles no código HTML (DOM).

- **Onde funciona:** Apenas em itens dentro de um container **Flexbox** (`display: flex`) ou **Grid** (`display: grid`).
- **Onde NÃO funciona:** Elementos normais (`block/inline`) que não estão em um container flex/grid.

2. Como funciona a lógica?



Imagine que cada elemento recebe um "número de chamada". Eles se organizam do menor para o maior (ordem crescente).

- **Valor Padrão:** Todos os itens começam com `order: 0`.
- **Números Negativos:** Vêm primeiro (ex: `-1` aparece antes de `0`).
- **Números Positivos:** Vêm depois (ex: `5` aparece depois de `2`).
- **Empate:** Se dois itens tiverem o mesmo número de `order`, ganha quem aparecer primeiro no código HTML.

3. Sintaxe

CSS

```
.item {  
  /* Aceita apenas números inteiros (positivos ou negativos) */  
  order: 2;  
  order: -1;  
  order: 0; /* Padrão */  
}
```

4. Exemplo Prático: Layout de Colunas

Imagine que no seu HTML a ordem é: **Artigo Principal** -> **Menu** -> **Propaganda**.

Mas visualmente, você quer: **Menu (Esquerda)** -> **Artigo (Meio)** -> **Propaganda (Direita)**.

HTML (Código Original):

HTML

```
<div class="container">  
  <article>Artigo (Código 1º)</article>  
  <nav>Menu (Código 2º)</nav>  
  <aside>Propaganda (Código 3º)</aside>  
</div>
```

CSS (A Mágica Visual):

CSS

```
.container {  
  display: flex;  
}  
  
nav {  
  order: 1; /* Aparece primeiro */  
}  
  
article {  
  order: 2; /* Aparece no meio */  
}  
  
aside {  
  order: 3; /* Aparece por último */  
}
```

⚠ 5. Acessibilidade (Muito Importante!)

Atenção: O `order` muda apenas o que os olhos veem, não o que os robôs leem.

Leitores de tela (usados por pessoas com deficiência visual) leem o site na ordem do **HTML**, ignorando o CSS `order`.

- **Não use** `order` para reordenar parágrafos de texto ou formulários, pois a leitura ficará confusa.
 - **Use** apenas para ajustes visuais de layout.
-

6. Ficha Técnica (Resumo Rápido)

Característica	Detalhe
Valor Inicial	0
Herdado?	Não
Animável?	Sim (dá para fazer transições suaves)
Suporte	Todos os navegadores modernos

Dica para o Caderno

Desenhe três caixinhas numeradas:

1. Caixa A (`order: 2`)
2. Caixa B (`order: -1`)
3. Caixa C (`order: 0`)

Resultado Visual: [B] -> [C] -> [A]

(Porque `-1` é menor que `0`, que é menor que `2`).

Com certeza! Vamos organizar isso direitinho para o seu caderno.

Separei exatamente como você pediu: primeiro o conceito do `align-self` (o "filho rebelde"), depois a diferença entre os três "aligns" e, por fim, a lista de valores sem usar tabelas.

Resumo de Estudos: CSS `align-self`

1. O que é?

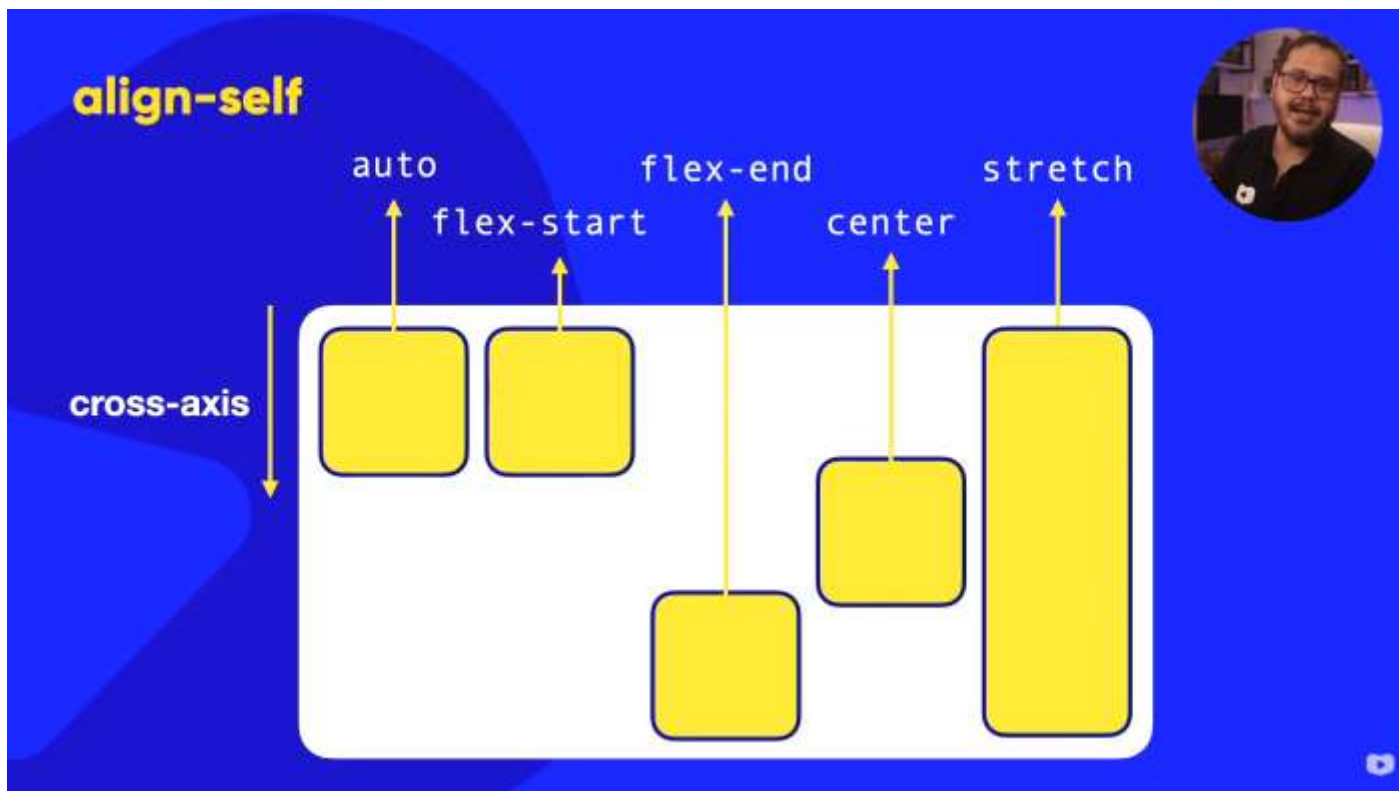
O `align-self` serve para alinhar **um único item** específico dentro de um container (Flex ou Grid).

Ele permite que esse item "desobedeça" a regra geral definida pelo pai (`align-items`) e tenha seu próprio alinhamento.

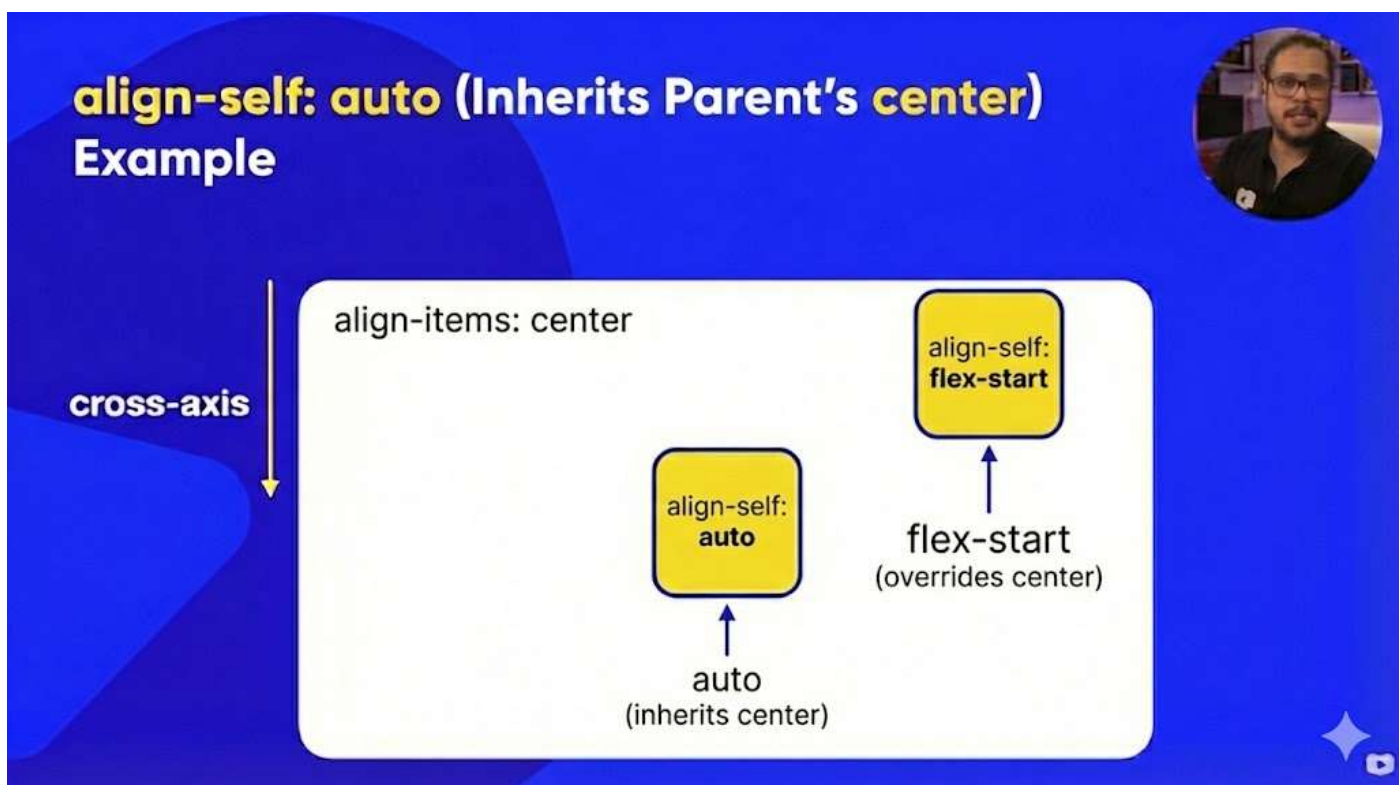
- **Eixo:** Funciona no eixo transversal (vertical, de cima para baixo na maioria dos casos).
- **Hierarquia:** Ele tem poder superior ao `align-items`.

Mnemônica: Pense nele como o "Selfie". É sobre **ele mesmo**, não sobre o grupo.

Valores do `align-self`



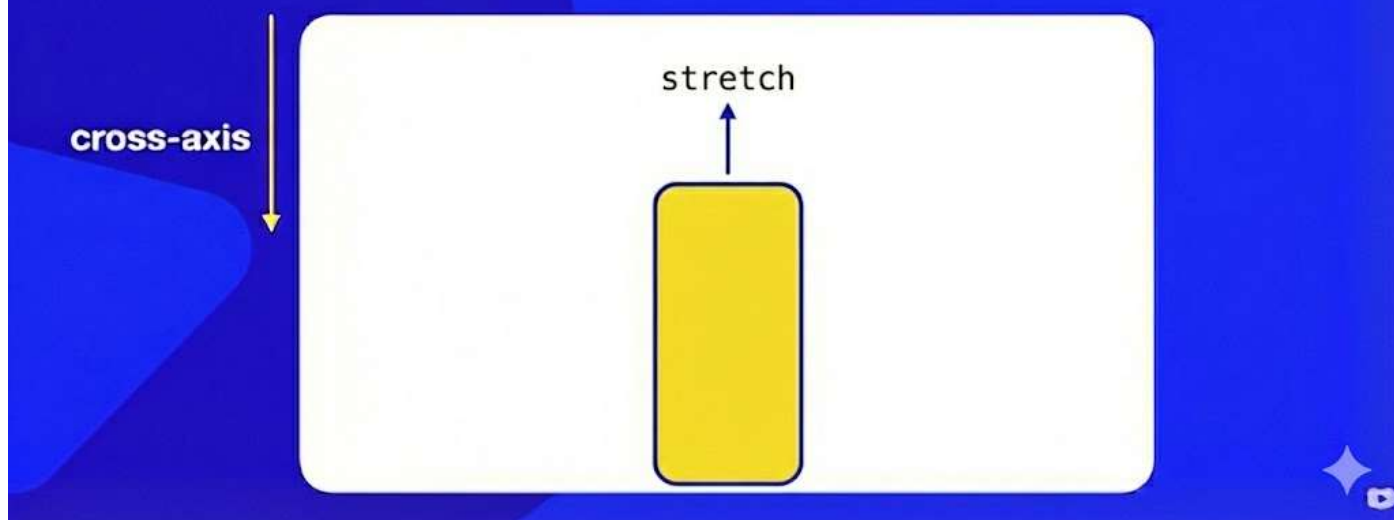
auto (O Padrão)



O item olha para o pai e diz: "O que você mandou no `align-items`? Vou seguir aquilo". Se o pai não tiver regra, ele age como `stretch`.

stretch (Esticar)

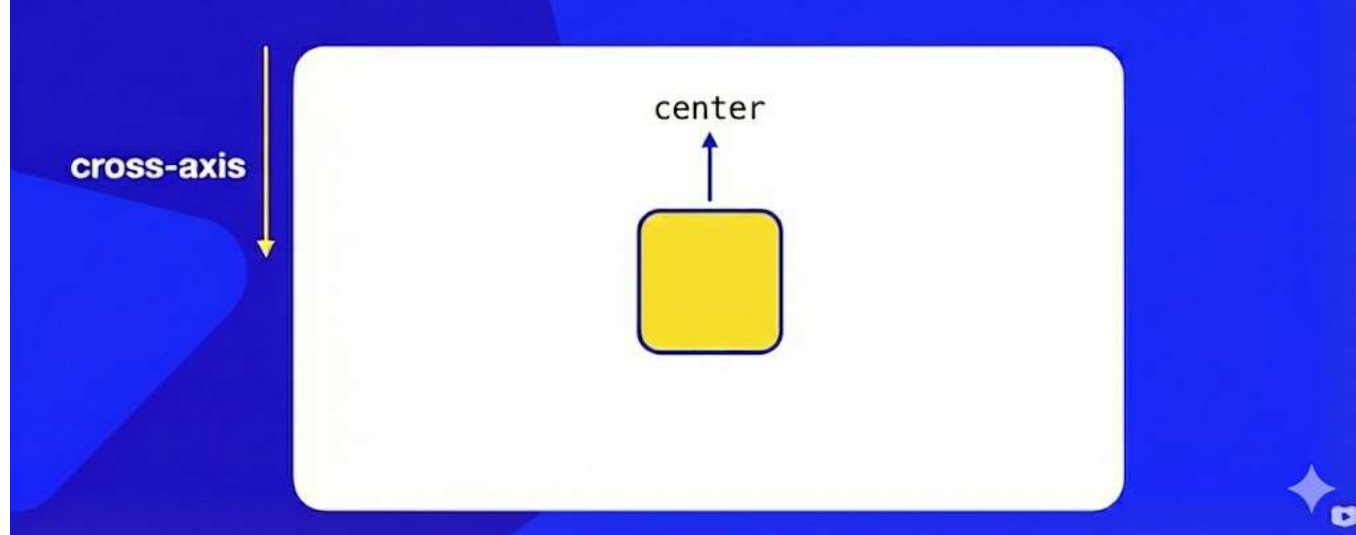
align-self: stretch Example



Se o item não tiver altura definida (`height`), ele estica para ocupar todo o espaço vertical disponível do container.

center (Centro do cross-axis)

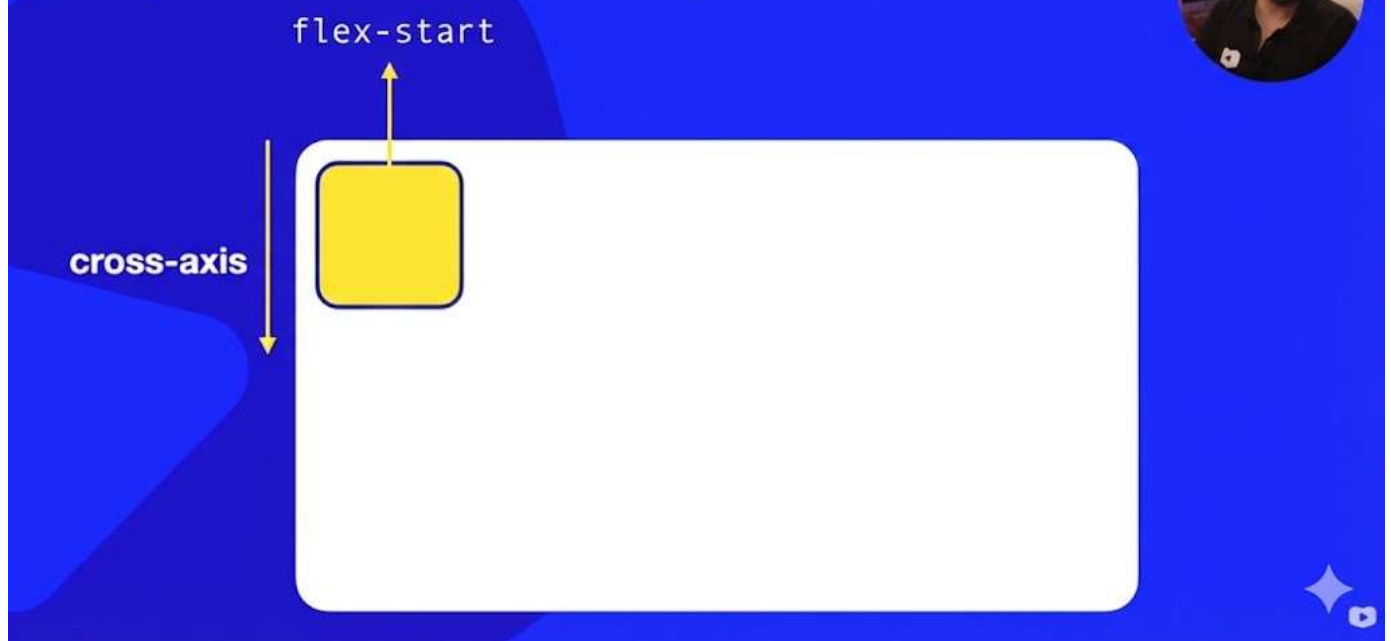
align-self: center Example



O item se posiciona exatamente no meio do eixo vertical.

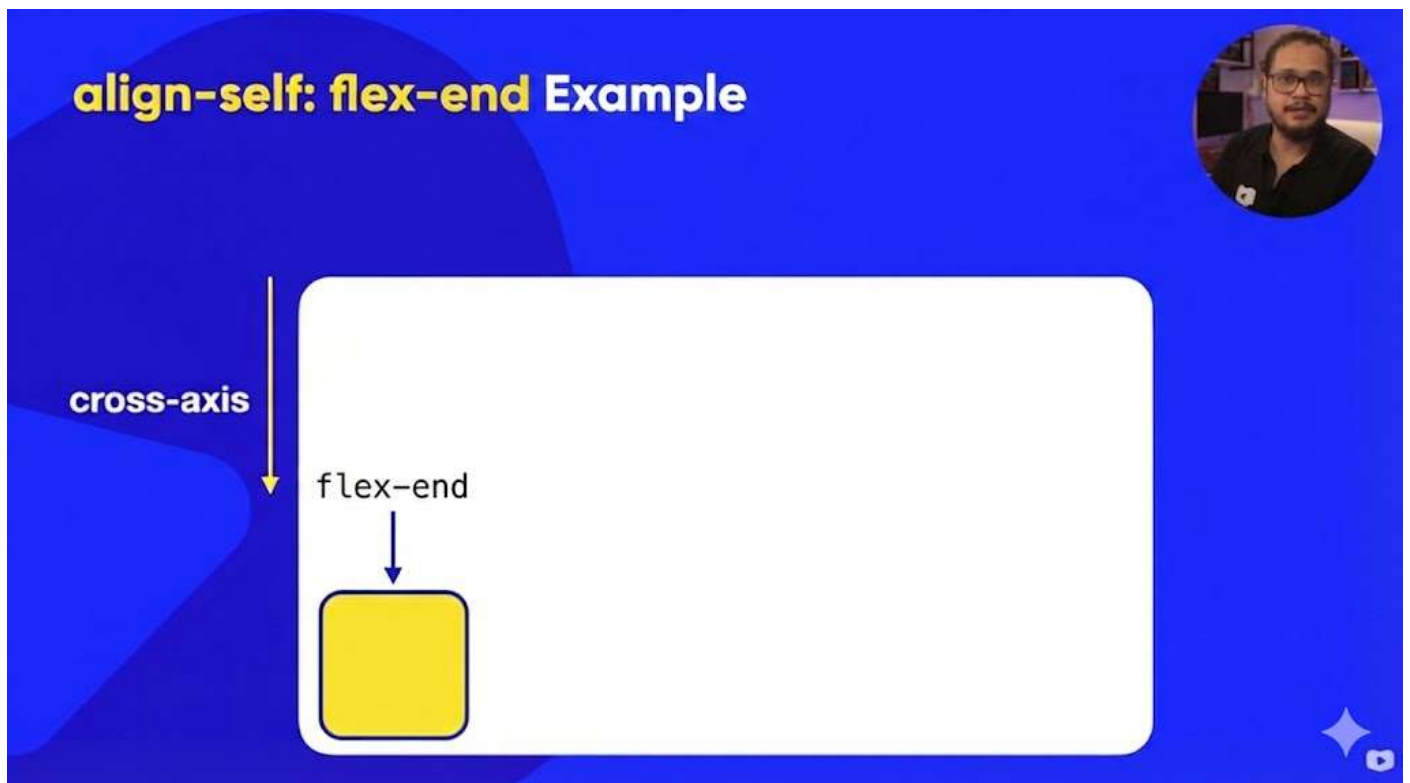
flex-start (cross-start)

align-self: flex-start Example



O item gruda no topo do container (ou onde começa o eixo transversal).

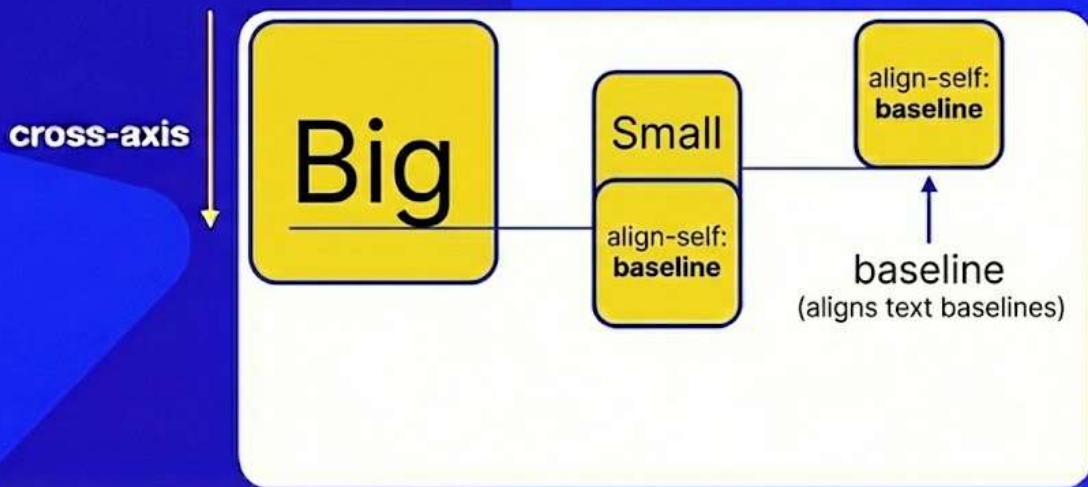
flex-end (cross-end)



O item gruda no fundo do container (ou onde termina o eixo transversal).

baseline (Linha do Texto)

align-self: baseline Example



O item se alinha baseando-se na linha onde o texto está escrito. Útil quando você tem itens com tamanhos de fonte diferentes e quer que o texto fique reto, não a caixa.

vs A Diferença: align-items vs align-self vs align-content

Aqui está a distinção fundamental para você não confundir mais:

1. align-items (O General)

- **Quem manda:** O Container (Pai).
- **O que faz:** Define a regra de alinhamento para **TODOS** os filhos de uma vez.
- *Exemplo:* "Todos vocês, fiquem no centro!"

2. align-self (O Rebelde)

- **Quem manda:** O Item (Filho).
- **O que faz:** Sobrescreve a regra do pai apenas para **AQUELE** elemento específico.
- *Exemplo:* "O pai mandou todos ficarem no centro, mas **eu** vou ficar no topo."

3. align-content (O Organizador de Linhas)

- **Quem manda:** O Container (Pai).
- **O que faz:** Só funciona quando o flexbox tem **várias linhas** (quebra de linha com `flex-wrap: wrap`). Ele alinha o *bloco de linhas* inteiro dentro do container.
- *Exemplo:* "Tenho 5 linhas de itens aqui, vou jogar todas essas linhas grudadas no final do container."

Exemplo de Código para o Caderno

CSS

```
.container {
  display: flex;
  align-items: center; /* Regra geral: todos no centro */
  height: 200px;
}

/* O item normal segue o pai (centro) */
.item-comum {
  background: blue;
}

/* O item rebelde vai para o topo */
.item-rebelde {
  align-self: flex-start; /* Ignora o center do pai */
  background: red;
}
```



CSS flex-basis

O Conceito Principal

A propriedade `flex-basis` define o **tamanho inicial** (ideal) de um item flexível antes que o espaço restante seja distribuído (esticado ou encolhido) pelas propriedades `flex-grow` e `flex-shrink`.

 **Ponto Chave:** Pense no `flex-basis` como o "ponto de partida". É o tamanho que o navegador considera para o elemento *antes* de fazer qualquer cálculo de layout flexível.

Eixo Principal (Main Axis)

O `flex-basis` não define necessariamente a largura (`width`). Ele define o tamanho no **Eixo Principal**.

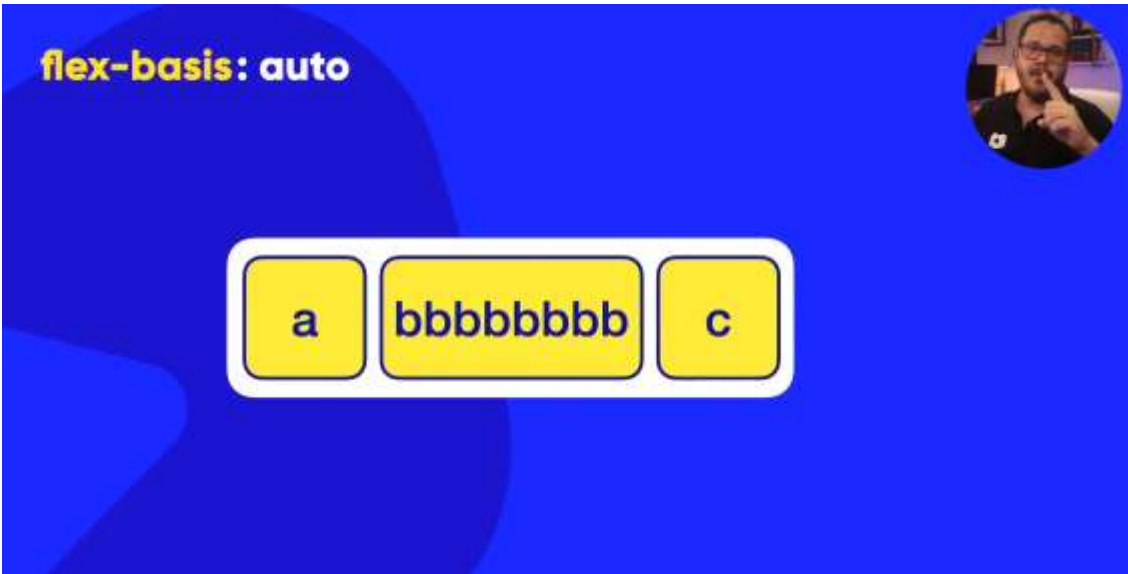
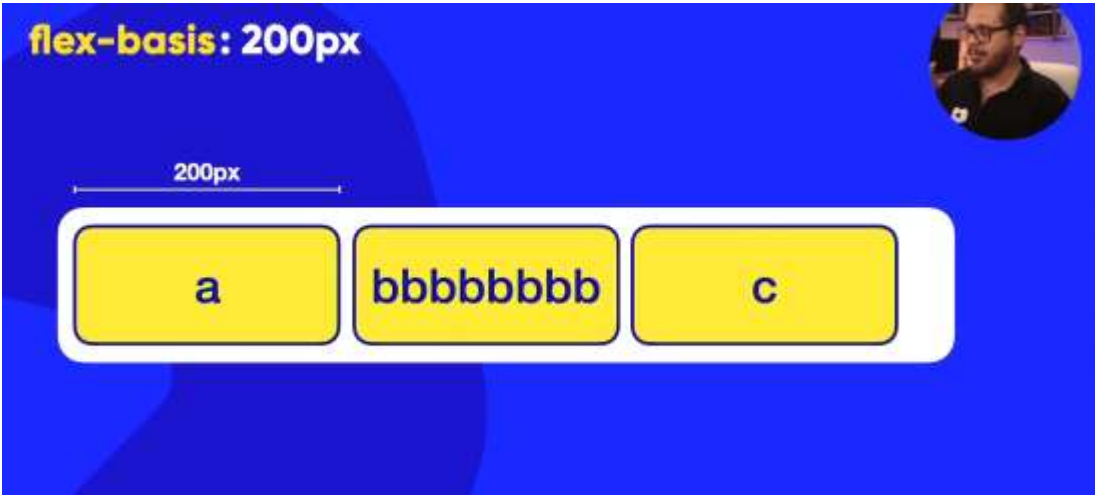
- Se `flex-direction: row` (padrão) → `flex-basis` controla a **Largura**.
 - Se `flex-direction: column` → `flex-basis` controla a **Altura**.
-

Sintaxe e Valores

CSS

```
/* Sintaxe Geral */
flex-basis: auto | content | <tamanho> | initial | inherit;
```

Valor	Descrição
auto	(Padrão) O navegador verifica se existe uma <code>width</code> (ou <code>height</code>) definida. Se sim, usa esse valor. Se não, usa o tamanho do conteúdo.

Valor	Descrição
	
<code>content</code>	O tamanho é baseado estritamente no conteúdo do item, ignorando qualquer <code>width</code> definida. (Suporte mais recente).
<code><tamanho></code>	Um valor específico de comprimento (<code>px</code>, <code>rem</code>, <code>em</code>, <code>%</code>, etc.). mas claro, eles vão até onde dá. Não significa que é 200px por exemplo que eles vão ser sempre 200px. <code>px</code>, <code>%</code>, <code>em</code>, etc. 
<code>0</code>	O item não ocupa espaço inicial. Muito usado para dividir espaços igualmente quando combinado com <code>flex-grow</code> .

VS Comparação: `flex-basis` VS `width`

Essa é a dúvida mais comum. Qual a diferença?

- Prioridade:** O `flex-basis` tem prioridade sobre o `width`. Se você definir `width: 200px` e `flex-basis: 100px`, o elemento começará com 100px.
- Restrição:** O `flex-basis` é apenas uma sugestão inicial. O `min-width` e `max-width` são limites rígidos e respeitados acima do `flex-basis`.

Tabela de Hierarquia de Tamanho:

1. `max-width / min-width` (Mandatários)
 2. `flex-basis` (Tamanho Flexível Inicial)
 3. `width` (Ignorado se `flex-basis` tiver um valor diferente de `auto`)
-

Exemplos Práticos

Exemplo 1: Tamanho Fixo Inicial

Aqui, o segundo item tenta ter 100px. Se houver espaço sobrando ou faltando, o `flex-grow` ou `flex-shrink` entrarão em ação a partir desses 100px.

CSS

```
.container {
  display: flex;
}

div:nth-of-type(2) {
  /* O item começará com 100px de largura */
  flex-basis: 100px;
}
```

Exemplo 2: A Mágica do `flex: 1`

Quando usamos a propriedade abreviada `flex: 1`, o navegador define automaticamente o `flex-basis` como 0%.

CSS

```
.item {
  /* Isso é o mesmo que: flex-grow: 1, flex-shrink: 1, flex-basis: 0% */
  flex: 1;
}
```

Por que 0%? Para garantir que todos os itens tenham tamanhos iguais, ignorando o tamanho do conteúdo que está dentro deles.



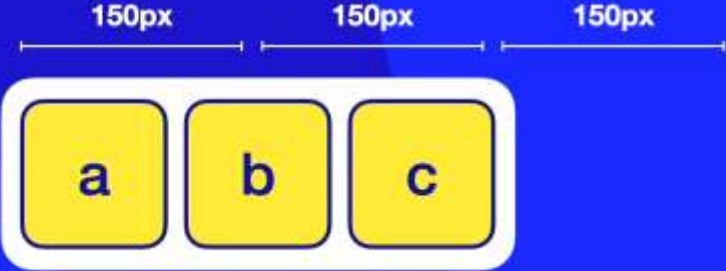
Notas para o Caderno

- **Propriedade Pai vs Filho:** O `flex-basis` deve ser aplicado no **item filho** (flex item), não no container pai.
 - **Animação:** Sim, é possível animar o `flex-basis` com CSS Transitions, criando efeitos suaves de expansão.
 - **Boas Práticas:** Geralmente, recomenda-se usar a propriedade abreviada `flex` (ex: `flex: 1 1 100px`) em vez de escrever `flex-basis` isoladamente, pois a abreviação define corretamente os valores de crescimento e encolhimento para evitar bugs de layout.
-

🚀 Flex-Grow e Flex-Shrink

1. flex-grow (O "Esticador")

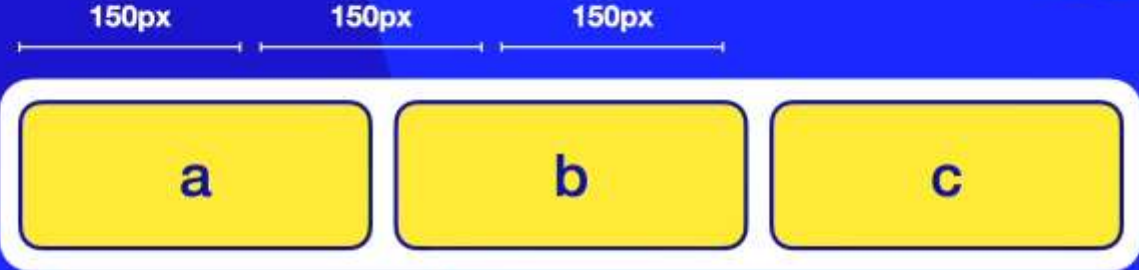
Pode encolher? Pode aumentar?



```
flex-shrink: 1  
flex-grow: 0
```

Small circular video inset of a man with glasses in the top right corner.

Pode encolher? Pode aumentar?



```
flex-shrink: 1  
flex-grow: 1
```

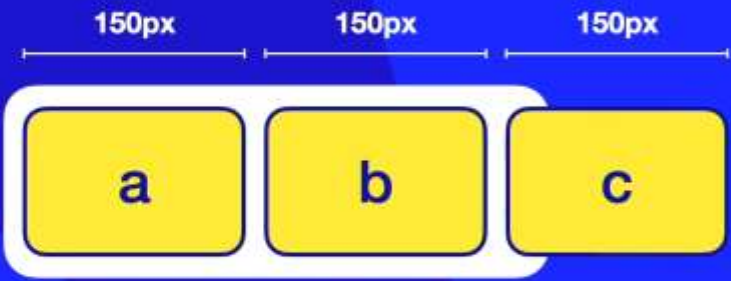
Small circular video inset of a man with glasses in the top right corner.

Define a capacidade de um item **crescer** se houver espaço livre no container. Ele decide como o espaço **vazio** será distribuído.

- **Valor Padrão:** 0 (Não cresça. Se sobrar espaço, deixe vazio).
- **Como funciona:** É uma proporção (ratio).

- 1– Pode crescer
- 0 – Não pode crescer

Pode encolher? Pode aumentar?

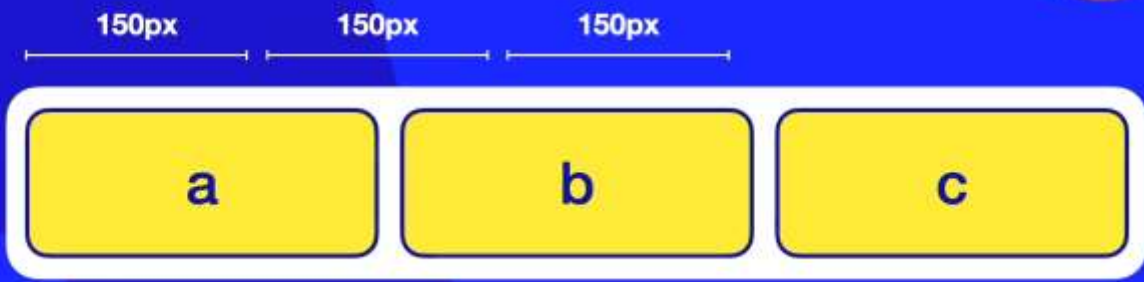


150px 150px 150px

a b c

```
flex-shrink: 0  
flex-grow: 1
```

Pode encolher? Pode aumentar?



150px 150px 150px

a b c

```
flex-shrink: 0  
flex-grow: 1
```

Ele pode ir mais de 1 ou 2..

Exemplos Práticos

Cenário A: Todos iguais

Se todos os itens tiverem `flex-grow: 1`, o espaço livre será dividido igualmente entre eles.
CSS

```
.item {  
  flex-grow: 1; /* Todos esticam por igual */  
}
```

Cenário B: O "Dominante"

Se um item tiver valor maior, ele pegará uma fatia maior do espaço livre.

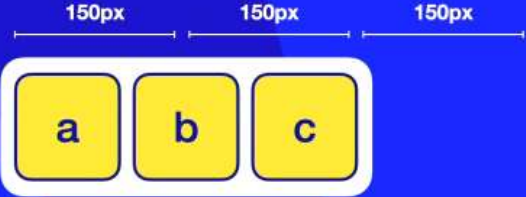
CSS

```
.item-normal {  
  flex-grow: 1;  
}  
  
.item-destaque {  
  flex-grow: 2; /* Vai tentar pegar o DOBRO do espaço livre dos outros */  
}
```

💡 **Nota:** O `flex-grow` não duplica o *tamanho total* do elemento, ele duplica a quantidade de *espaço extra* que é adicionado ao `flex-basis` dele.

3. `flex-shrink` (O "Encolhedor")

Pode encolher? Pode aumentar?



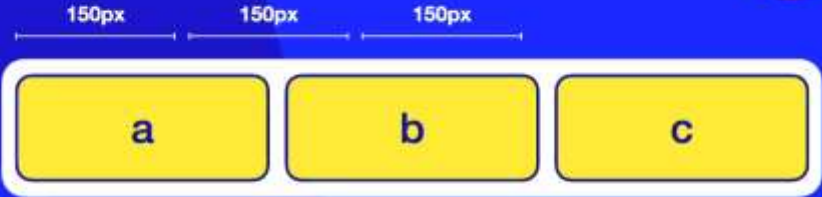
150px 150px 150px

a b c

`flex-shrink: 1`
`flex-grow: 0`



Pode encolher? Pode aumentar?



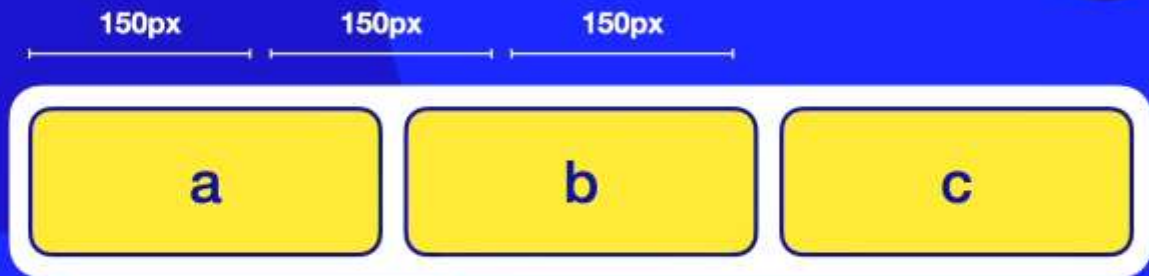
150px 150px 150px

a b c

`flex-shrink: 0`
`flex-grow: 1`



Pode encolher? Pode aumentar?



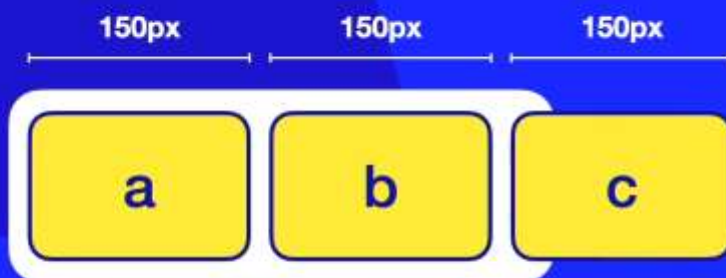
```
flex-shrink: 1  
flex-grow: 1
```



Define a capacidade de um item **encolher** se o container for menor que a soma dos itens.

- **Valor Padrão:** 1 (Sim, encolha se precisar para não quebrar o layout).
- **Como funciona:** Quanto maior o número, mais o item "emagrece" para salvar os outros.
- 1 – Pode escolher
- 0 – Não pode escolher

Pode encolher? Pode aumentar?



```
flex-shrink: 0  
flex-grow: 1
```



O Uso Mais Importante: `flex-shrink: 0`

Este é o uso mais comum no dia a dia. Imagine que você tem uma lista com uma foto de perfil e um texto. Você não quer que a foto fique "esmagada" se a tela for pequena.

CSS

```
.avatar {  
  width: 50px;           /* ou flex-basis: 50px */  
  flex-shrink: 0;        /* PROIBIDO encolher. Mantenha 50px custe o que custar! */  
}  
  
.texto {  
  flex-shrink: 1;        /* O texto pode encolher se faltar espaço */  
}
```

- **`flex-shrink: 1 (Padrão)`**: O item cede espaço.
- **`flex-shrink: 0`**: O item é rígido, não diminui nem um pixel abaixo do seu `flex-basis`.
- **`flex-shrink: 2+`**: O item se sacrifica e encolhe mais rápido que os outros.

3. Resumo Visual para o Caderno

Imagine o Container Flex como um vagão de metrô:

1. **`flex-basis`**: É o tamanho do corpo da pessoa (o espaço que ela ocupa sentada normalmente).
2. **`flex-grow`**: É quando o vagão está vazio. As pessoas podem esticar as pernas e ocupar os bancos vazios?
 - 0: Não (ficam comportadas).
 - 1: Sim (se espalham).
3. **`flex-shrink`**: É quando o vagão está lotado (hora do rush). Quem se aperta para caber mais gente?
 - 1: Todo mundo se aperta um pouco.
 - 0: A pessoa se recusa a se apertar (ocupa o espaço dela e empurra os outros).

4. A Propriedade Atalho: `flex` (A Recomendada)

Profissionais raramente escrevem `flex-grow`, `flex-shrink` e `flex-basis` separados. Usamos o atalho `flex`.



Sintaxe:

`flex: [grow] + [shrink] + [basis]`

Código	Significado	Tradução
<code>flex: 1;</code>	1 1 0%	Cresça, encolha e comece do zero (tamanhos iguais).
<code>flex: auto;</code>	1 1 auto	Cresça, encolha, mas respeite o tamanho do conteúdo inicial.
<code>flex: none;</code>	0 0 auto	Tamanho fixo. Não cresce nem encolhe.
<code>flex: 0 0 200px;</code>	0 0 200px	Fixo em 200px rígidos. Não muda nunca.



flex = flex-grow + flex-shrink + flex-basis

flex: 0 1 auto
flex: initial
flex: 0 0 auto
flex: none
flex: 1 1 auto
flex: auto
flex: 3
flex: 3 1 auto

🔧 Valores mais importantes (DECORA ISSO)

◆ Valor padrão do Flexbox

`flex: 0 1 auto;`

✚ Significa:

- Não cresce
- Pode encolher
- Tamanho depende do conteúdo

Equivalente a:

`flex: initial;`

◆ Item não flexível (rígido)

`flex: 0 0 auto;`

Ou simplesmente:

`flex: none;`

✦ Comportamento:

- ✗ Não cresce
- ✗ Não encolhe
- 📏 Tamanho fixo baseado no conteúdo

👉 Usado quando você **não quer que o item se adapte**.

◆ Item totalmente flexível

`flex: 1 1 auto;`

Ou:

`flex: auto;`

✦ Comportamento:

- ☒ Cresce
- ☒ Encolhe
- 📏 Baseado no conteúdo

👉 Muito usado para layouts responsivos.

12/34 Quando usar apenas um número

`flex: 3;`

Isso significa:

```
flex-grow: 3;  
flex-shrink: 1;  
flex-basis: auto;
```

✦ Regra importante:

Quando usamos **apenas um valor**, ele se aplica ao **flex-grow**.

Exemplos equivalentes:

```
flex: 1;  ≡  flex: 1 1 auto;  
flex: 2;  ≡  flex: 2 1 auto;  
flex: 3;  ≡  flex: 3 1 auto;
```

Exemplos práticos para o caderno

◆ Todos os itens com o mesmo tamanho

```
.item {  
  flex: 1;  
}
```

👉 Todos crescem igualmente, **independente do conteúdo**.

◆ Item fixo + item flexível

```
.menu {  
  flex: none;  
}  
  
.conteudo {  
  flex: 1;  
}
```

👉 O menu fica fixo, o conteúdo ocupa o resto da tela.

◆ Item que não encolhe

```
.item {  
  flex: 1 0 auto;  
}
```

👉 Cresce, mas **não diminui**.

Resumo mental (anota isso)

```
flex: initial  →  0 1 auto  (padrão)  
flex: none     →  0 0 auto  (rígido)  
flex: auto     →  1 1 auto  (totalmente flexível)
```